

Apprendre à programmer (avec Visual Basic)

**Jean-Christophe RANGON
2026**



email: `jc.rangon.formateur@gmail.com`

linkedIn: `www.linkedin.com/in/jean-christophe-rangon-dev-web`

SUPPORT FORMATEUR ENRICHI

Visual Basic .NET - Jour 1 - Cours orienté LIVE CODING

Du besoin au premier programme : algorithmique, Visual Studio, variables, types, conversions, opérateurs et lisibilité.

Repère	Contenu
Formation	Apprendre à programmer avec Visual Basic .NET
Durée J1	7 heures - 09h00-12h30 / 13h30-17h00
Public	Débutants en programmation
Mode	Live coding majoritaire - environ 70 à 80 % éditeur / exécution / diagnostic
Fil rouge	Mini-application de gestion de contacts
Production	Premier programme, fiche Contact console et calculateur console

Principe directeur - Le support sert de référence de fond. En salle, la notion est d'abord provoquée par une question ou une expérience, puis nommée et structurée après observation.

1. Comment utiliser ce support pendant une journée live coding

Ce document n'est pas un script à lire mot à mot. Il contient la matière complète du J1, les explications à donner, les expériences à mener dans Visual Studio, les erreurs utiles à provoquer, les mini-défis, les variantes et les versions finales exécutables. Le rythme de cours doit rester actif : peu de temps entre une idée et son exécution.

1.1 La boucle pédagogique de référence

Étape	Rôle du formateur	Rôle des étudiants
1. Besoin	Poser un problème concret sans montrer la syntaxe.	Reformuler ce qu'il faut obtenir.
2. Prédiction	Demander ce que le programme devrait faire.	Proposer une sortie, un type, une règle.
3. Code progressif	Taper une seule nouveauté.	Suivre, dicter, compléter.
4. Exécution	Compiler/lancer immédiatement.	Observer et comparer à la prédiction.
5. Diagnostic	Faire lire le message ou l'état.	Expliquer ce qui s'est passé.
6. Variation	Changer un seul élément.	Prédire l'impact.
7. Mini-défi	Donner 3 à 8 minutes d'autonomie.	Modifier ou reconstruire.
8. Synthèse	Nommer la règle et l'écrire.	Reformuler avec leurs mots.

[RÈGLE] Ne pas taper 25 lignes d'un coup. Une exécution fréquente rend visibles les liens entre cause et effet.

1.2 Préparation technique avant 09h00

- Visual Studio installé avec la charge de travail Développement .NET Desktop.
- Un projet Console App Visual Basic peut être créé sans téléchargement supplémentaire.
- Police de l'éditeur agrandie pour projection (environ 16-20 pt selon salle).
- Fenêtres Solution Explorer, Error List et Output repérées.
- Dossier de travail vide prêt pour les mini-projets du J1.
- Option Strict On recommandée pour rendre les conversions explicites et pédagogiques.

1.3 Arborescence conseillée des démonstrations

```
J1/
  01_HelloConsole/
  02_Prenom/
  03_ErreursCompilation/
  04_TypesConversions/
```

```
05_Operateurs/
06_RefactorisationLisibilite/
07_ContactConsole/
08_TP_Calculateur/
```

2. Objectifs pédagogiques du Jour 1

- Distinguer clairement un besoin, un algorithme, une syntaxe et une exécution.
- Créer et exécuter une application console Visual Basic .NET dans Visual Studio.
- Lire une première erreur de compilation et utiliser le message comme indice.
- Déclarer des variables et constantes avec des types simples adaptés.
- Comprendre que Console.ReadLine retourne du texte et qu'une conversion peut être nécessaire.
- Utiliser Integer, Double, Decimal, String, Boolean et Date dans des exemples accessibles.
- Prédire puis vérifier des expressions arithmétiques, comparatives et logiques.
- Améliorer un code fonctionnel mais peu lisible sans en changer le comportement.
- Construire une première fiche Contact en console.
- Réaliser un calculateur console simple en autonomie guidée.

[POINT DE CONTRÔLE FINAL] À 17h00, l'apprenant doit savoir partir d'un besoin court, écrire quelques étapes en pseudo-code, les traduire en VB, choisir un type simple, convertir une saisie et expliquer une expression.

2.1 Carte mentale de la journée

Le fil logique est volontairement linéaire : on part du problème, on décrit les étapes, on découvre l'éditeur, puis on donne des noms et des types aux données. Les opérateurs permettent de transformer ou comparer ces données. Enfin, on améliore la lisibilité et on assemble le tout dans une petite production.

```
BESOIN
↓
ALGORITHME / PSEUDO-CODE
↓
SYNTAXE VISUAL BASIC
↓
COMPILATION
↓
EXÉCUTION
↓
DONNÉES → TYPES → CONVERSIONS → OPÉRATEURS
↓
CODE LISIBLE
↓
MINI-APPLICATION CONSOLE
```

3. 09h00-09h25 - Accueil, diagnostic et environnement

3.1 Question d'ouverture : « Qu'est-ce qu'un programme ? »

Éviter de donner immédiatement une définition académique. Demander trois propositions, noter quelques mots au tableau, puis converger vers une définition simple : un programme est une suite d'instructions qu'un ordinateur peut exécuter pour réaliser un traitement.

[QUESTION] « Quand vous double-cliquez sur une application, qu'est-ce que l'ordinateur exécute réellement ? »

[RÉPONSE ATTENDUE] Un programme compilé / un exécutable ou, plus généralement, du code transformé dans une forme exécutable par la plateforme.

3.2 Première expérience immédiate

Créer un projet Console App Visual Basic nommé J1_Diagnostic. Ne pas expliquer encore toutes les lignes. Le but est simplement de montrer qu'un fichier contient du code, qu'on peut l'exécuter et observer un résultat.

```
Module Program
    Sub Main(args As String())
        Console.WriteLine("Bonjour !")
    End Sub
End Module
```

[PRÉDICTION] Avant de lancer : « Qu'allez-vous voir ? Où pensez-vous que le texte va apparaître ? »

[EXÉCUTION] Lancer avec Ctrl+F5. Faire distinguer l'éditeur (où l'on écrit) et la console (où le programme s'exécute).

3.3 Repères Visual Studio à montrer, sans tout expliquer

Zone	À quoi elle sert aujourd'hui
Solution Explorer	Voir le projet et les fichiers.
Éditeur	Écrire et modifier le code.
Barre de lancement	Démarrer l'application.
Error List	Lire les erreurs de compilation.
Output	Observer les messages de construction si nécessaire.
Console	Voir les entrées/sorties de l'application console.

3.4 Diagnostic rapide

Afficher ces quatre fragments l'un après l'autre. Le diagnostic ne cherche pas à noter les étudiants ; il sert à voir comment ils raisonnent.

```
Console.WriteLine("5 + 2")
```

```
Console.WriteLine(5 + 2)
Dim age As Integer = 20
Console.WriteLine(age > 18)
```

1. Demander ce que chaque ligne affiche.
2. Ne corriger qu'après le vote du groupe.
3. Exécuter.
4. Faire expliciter la différence entre texte, calcul et comparaison.

4. 09h25-10h20 - De l'algorithme au code

4.1 Le besoin avant la syntaxe

Besoin : demander le prénom d'une personne puis afficher « Bonjour <prénom> ». Avant d'écrire du Visual Basic, faire verbaliser les étapes.

[TABLEAU] Écrire trois colonnes : ENTRÉE / TRAITEMENT / SORTIE. Puis placer « prénom » dans Entrée et « message Bonjour » dans Sortie.

4.2 Définition simple puis rigoureuse

Définition simple : un algorithme est une suite d'étapes ordonnées pour résoudre un problème. Définition plus rigoureuse : c'est une procédure finie, non ambiguë, décrivant des opérations et leur ordre pour transformer des entrées en résultats.

[À DIRE] « L'algorithme ne dépend pas de Visual Basic. On peut décrire la même solution avant de choisir le langage. »

4.3 Premier pseudo-code

```
DÉBUT
Afficher "Quel est votre prénom ?"
Lire prenom
Afficher "Bonjour " + prenom
FIN
```

4.4 Traduction progressive en Visual Basic

Étape 1 - uniquement afficher la question :

```
Console.WriteLine("Quel est votre prénom ?")
```

[EXÉCUTION] Lancer. Rien n'est encore lu. Demander ce qui manque.

Étape 2 - lire du texte :

```
Dim prenom As String = Console.ReadLine()
```

[À DIRE] « Dim crée une variable. String indique que cette variable contient du texte. Console.ReadLine lit ce que l'utilisateur tape. »

Étape 3 - réutiliser la donnée :

```
Console.WriteLine("Bonjour " & prenom)
```

4.5 Pourquoi utiliser & pour concaténer ?

En Visual Basic, & exprime clairement la concaténation de chaînes. Le signe + peut aussi être rencontré, mais & est plus explicite pour assembler du texte et évite certaines ambiguïtés pédagogiques avec l'addition.

[VARIATION] Changer « Bonjour » en « Bienvenue ». Puis ajouter « ! ». Enfin tester un prénom vide.

4.6 Version complète exécutable

```
Module Program
    Sub Main(args As String())
        Console.WriteLine("Quel est votre prénom ?")
        Dim prenom As String = Console.ReadLine()
        Console.WriteLine("Bonjour " & prenom & " !")
    End Sub
End Module
```

4.7 Mini-défi

Sans montrer la solution : demander le nom et la ville, puis afficher une phrase du type « Alice habite à Lyon ». Laisser 5 minutes, puis corriger en faisant dicter les lignes par les étudiants.

5. 10h35-11h35 - Premier programme Visual Basic : comprendre ce que l'on tape

5.1 Module, Sub Main et instructions

Ne pas entrer dans une théorie complète des modules. Donner seulement le modèle mental utile : Visual Basic organise le code dans des structures. Dans une application console classique, Main est le point de départ. Une instruction est une action demandée au programme.

Élément	Explication J1
Module Program	Conteneur de code généré par le modèle de projet.
Sub Main(...)	Point de départ de l'application console.
Console.WriteLine	Écrit une ligne dans la console.
Console.ReadLine	Lit une ligne saisie par l'utilisateur.
End Sub	Fin de la procédure Main.
End Module	Fin du module.

5.2 Compiler n'est pas exécuter

La compilation vérifie et transforme le code source. L'exécution lance le programme résultant. Pour débiter, l'idée essentielle est qu'une erreur de syntaxe peut empêcher la compilation avant même que le programme ne démarre.

[QUESTION] « Si j'oublie une parenthèse, le programme peut-il commencer puis planter plus tard ? »

[RÉPONSE ATTENDUE] En général ici, non : la compilation échoue avant le lancement.

5.3 Erreur volontaire n°1 - parenthèse manquante

```
Console.WriteLine("Bonjour"
```

[DIAGNOSTIC] Ne pas corriger immédiatement. Montrer le soulignement et la liste d'erreurs. Faire lire le message puis demander où se trouve l'indice utile.

5.4 Erreur volontaire n°2 - membre inexistant

```
Console.WritLine("Bonjour")
```

Faire repérer la faute de frappe. Insister : le compilateur ne « comprend pas l'intention » ; il vérifie des symboles précis.

5.5 Erreur volontaire n°3 - chaîne non terminée

```
Console.WriteLine("Bonjour)
```

[RÈGLE] Le message du compilateur n'est pas une punition ; c'est une information de diagnostic. Lire avant de modifier.

5.6 Imports et bibliothèques : niveau J1

Expliquer simplement qu'un programme s'appuie sur une bibliothèque .NET contenant des types déjà écrits. Console fait partie de ces fonctionnalités. Imports permet de rendre certains noms accessibles plus simplement. Ne pas approfondir les assemblies au-delà de cette intuition.

6. 11h35-12h30 - Variables : donner un nom à une donnée

6.1 Modèle mental

Une variable est un emplacement nommé utilisé pour mémoriser une valeur qui peut changer pendant l'exécution. Le type indique la nature des valeurs que l'on souhaite y stocker et les opérations cohérentes avec ces valeurs.

[ANALOGIE] Une boîte étiquetée « age » contenant un entier. L'étiquette est le nom ; la forme de la boîte représente le type ; le contenu est la valeur.

6.2 Déclaration, initialisation, affectation

```
Dim age As Integer
age = 25
```

```
Dim prenom As String = "Nora"
```

Dans le premier cas, la déclaration et l'affectation sont séparées. Dans le second, la variable est déclarée et initialisée en une seule instruction.

6.3 Les types simples utiles aujourd'hui

Type	Exemple	Quand l'utiliser
Integer	42	Nombre entier : âge, quantité, compteur.
Double	1.75	Nombre décimal scientifique/général.
Decimal	19.99D	Montants et calculs décimaux nécessitant une bonne précision décimale.
String	"Paris"	Texte.
Boolean	True	Vrai/faux, état logique.
Date	#2026-09-27#	Date et heure.

[PRÉDICTION] Que se passe-t-il si on écrit `Dim age As Integer = "bonjour"` avec `Option Strict On` ?

[RÉPONSE ATTENDUE] La valeur texte ne peut pas être convertie implicitement en Integer : erreur de compilation.

6.4 Constantes

```
Const TauxTVA As Decimal = 0.2D
```

Une constante porte un nom comme une variable, mais sa valeur n'est pas censée changer pendant l'exécution. L'intérêt n'est pas seulement technique : elle rend l'intention visible et évite les « nombres magiques ».

6.5 Mini-expériences de type

- Créer `age As Integer` puis afficher `age`.
- Remplacer Integer par Double et utiliser 25.5.
- Créer `estMajeur As Boolean = True`.
- Créer `prix As Decimal = 19.9D`.
- Demander aux étudiants ce qui change : syntaxe, valeur possible, opérations pertinentes.

7. Lire une saisie : pourquoi tout arrive d'abord comme du texte

7.1 Console.ReadLine retourne une chaîne

Même si l'utilisateur tape 25, la console reçoit les caractères « 2 » et « 5 ». Pour effectuer une addition numérique, il faut convertir cette chaîne en nombre.

```
Console.Write("Votre âge : ")
Dim saisie As String = Console.ReadLine()
Console.WriteLine(saisie)
```

[QUESTION] « Si je tape 25, est-ce déjà un Integer ? »

[RÉPONSE ATTENDUE] Non. ReadLine renvoie du texte.

7.2 Parse : conversion directe

```
Dim age As Integer = Integer.Parse(Console.ReadLine())
```

Parse est simple mais suppose que le texte représente bien un entier. Si l'utilisateur tape « vingt », une exception survient à l'exécution.

7.3 Expérience : provoquer l'échec de Parse

[ERREUR VOLONTAIRE] Lancer le programme puis saisir abc. Ne pas masquer l'exception. Faire identifier : le programme a compilé, puis a échoué pendant l'exécution.

C'est l'occasion de distinguer trois familles qui seront approfondies en J3 : erreur de compilation, erreur d'exécution, erreur logique.

Famille	Exemple J1	Moment
Compilation	Parenthèse manquante	Avant l'exécution.
Exécution	Integer.Parse("abc")	Pendant l'exécution.
Logique	Additionner quand on voulait soustraire	Le programme tourne mais le résultat est faux.

7.4 TryParse : tenter sans provoquer d'exception

```
Console.Write("Votre âge : ")
Dim saisie As String = Console.ReadLine()
Dim age As Integer
Dim conversionOk As Boolean = Integer.TryParse(saisie, age)

Console.WriteLine("Conversion réussie ? " & conversionOk)
Console.WriteLine("Âge : " & age)
```

TryParse renvoie un Boolean indiquant si la conversion a réussi et place la valeur convertie dans la variable de sortie. À ce stade, on peut montrer le mécanisme sans encore introduire un If complet ; les conditions seront travaillées en J2.

[MINI-DÉFI] Lire un prix en Decimal avec Decimal.TryParse, puis afficher le booléen de réussite et la valeur obtenue.

8. 13h30-14h20 - Opérateurs arithmétiques et priorité

8.1 Une expression produit une valeur

Une expression combine des valeurs, variables et opérateurs pour produire un résultat. Avant chaque exécution, demander aux étudiants de prédire ce résultat.

8.2 Opérateurs arithmétiques

Opérateur	Sens	Exemple
+	Addition	$10 + 3 \rightarrow 13$
-	Soustraction	$10 - 3 \rightarrow 7$
*	Multiplication	$10 * 3 \rightarrow 30$
/	Division réelle	$10 / 4 \rightarrow 2.5$
\	Division entière	$10 \setminus 4 \rightarrow 2$
Mod	Reste de division	$10 \text{ Mod } 4 \rightarrow 2$
^	Puissance	$2 ^ 3 \rightarrow 8$

8.3 Live coding de prédiction

```
Console.WriteLine(2 + 3 * 4)
Console.WriteLine((2 + 3) * 4)
Console.WriteLine(10 / 4)
Console.WriteLine(10 \ 4)
Console.WriteLine(10 Mod 4)
```

[PRÉDICTION] Faire voter pour chaque sortie avant de lancer.

La priorité des opérateurs explique pourquoi multiplication/division sont évaluées avant addition/soustraction. Les parenthèses rendent l'ordre explicite et améliorent souvent la lisibilité.

8.4 Mini-défis arithmétiques

- Calculer le prix TTC à partir d'un prix HT et d'un taux.
- Calculer une moyenne de trois notes.
- Convertir des minutes en heures + minutes avec division entière et Mod.

- Calculer le périmètre et l'aire d'un rectangle.

8.5 Cas monétaire avec Decimal

```
Dim prixHT As Decimal = 100D
Const tauxTVA As Decimal = 0.2D
Dim prixTTC As Decimal = prixHT * (1D + tauxTVA)
Console.WriteLine(prixTTC)
```

[VARIATION] Changer prixHT et tauxTVA, puis demander si une constante convient toujours pour le taux dans ce petit programme.

9. Comparaisons et expressions booléennes

9.1 Comparer produit un Boolean

```
Dim age As Integer = 20
Console.WriteLine(age >= 18)
Console.WriteLine(age = 20)
Console.WriteLine(age <> 20)
```

Le résultat n'est pas « oui » ou « non » mais une valeur booléenne True ou False.

9.2 Comparateurs à connaître

Opérateur	Question
=	Les deux valeurs sont-elles égales ?
<>	Sont-elles différentes ?
<	La gauche est-elle plus petite ?
>	La gauche est-elle plus grande ?
<=	Plus petite ou égale ?
>=	Plus grande ou égale ?

9.3 And, Or et Not

Ces opérateurs combinent ou inversent des booléens. Les conditions If seront vues en J2 ; aujourd'hui, l'objectif est de savoir lire et prédire une expression.

```
Dim age As Integer = 20
Dim aPermis As Boolean = True

Console.WriteLine(age >= 18 And aPermis)
Console.WriteLine(age < 18 Or Not aPermis)
```

```
Console.WriteLine(Not (age >= 18))
```

9.4 Tables de vérité simplifiées

A	B	A And B	A Or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

[QUESTION] « Pour entrer dans une zone, il faut être majeur ET avoir un badge. Quel opérateur représente cette règle ? »

[QUESTION] « Pour bénéficier d'une remise, il suffit d'être étudiant OU senior. Quel opérateur ? »

9.5 Mini-défi logique

Créer age, estEtudiant et aInvitation. Écrire puis tester une expression « accès autorisé si majeur et (étudiant ou invité) ». Faire varier une seule valeur à la fois.

```
Dim acces As Boolean = age >= 18 And (estEtudiant Or aInvitation)
```

10. Affectation versus égalité : éviter une confusion de débutant

En Visual Basic, le symbole = apparaît à la fois dans une affectation et dans une comparaison, selon le contexte. C'est le contexte syntaxique qui indique l'intention.

```
Dim age As Integer
age = 25      ' affectation
Console.WriteLine(age = 25) ' comparaison
```

[À DIRE] « À gauche d'une affectation, on désigne l'endroit où ranger la nouvelle valeur. Dans une expression, = pose la question : est-ce égal ? »

10.1 Expérience de lecture

10. Afficher les deux lignes séparément.
11. Demander laquelle modifie la variable.
12. Demander laquelle produit True/False.
13. Faire reformuler la règle.

10.2 Pourquoi ne pas tout introduire aujourd'hui

Les structures If et Select Case seront étudiées en J2. Aujourd'hui, on veut que l'étudiant sache construire la valeur booléenne qui servira demain à décider.

[TRANSITION J2] « Aujourd'hui nous savons fabriquer une question vraie/fausse. Demain, nous apprendrons au programme quoi faire selon la réponse. »

11. 14h20-15h15 - Lisibilité : refactoriser sans changer le comportement

11.1 Le code peut fonctionner et rester mauvais à lire

La lisibilité réduit le coût de compréhension. Un nom explicite, une indentation régulière et quelques commentaires utiles permettent de comprendre plus vite l'intention. L'objectif de la séquence est de modifier la forme tout en vérifiant que le comportement reste identique.

11.2 Version volontairement fragile

```
Module Program
Sub Main(args As String())
Dim x As Decimal=100
Dim y As Decimal=0.2D
Dim z=x*(1+y)
Console.WriteLine("r=" & z)
End Sub
End Module
```

[QUESTION] « Le programme fonctionne. Pourquoi est-il néanmoins coûteux à relire ? »

11.3 Refactorisation étape par étape

Étape 1 - noms :

```
Dim prixHT As Decimal = 100D
Dim tauxTVA As Decimal = 0.2D
Dim prixTTC As Decimal = prixHT * (1D + tauxTVA)
```

Étape 2 - constante :

```
Const tauxTVA As Decimal = 0.2D
```

Étape 3 - sortie explicite :

```
Console.WriteLine("Prix TTC : " & prixTTC & " €")
```

Après chaque étape, exécuter. Faire constater que le comportement métier reste identique alors que l'intention devient plus visible.

11.4 Les commentaires utiles

Un commentaire utile explique une intention non évidente, une règle métier ou une raison. Il ne doit pas répéter mot pour mot le code.

```
Const tauxTVA As Decimal = 0.2D ' Taux utilisé pour l'exercice pédagogique
```

[CONTRE-EXEMPLE] ' On crée une variable prixHT juste au-dessus de Dim prixHT... n'apporte presque rien.

11.5 Checklist de lisibilité J1

- Nommer les variables d'après leur rôle.
- Mettre des espaces autour des opérateurs.
- Indenter les blocs automatiquement.
- Éviter les abréviations opaques.
- Utiliser Const pour une valeur fixe significative.
- Commenter le pourquoi plutôt que le quoi.
- Exécuter après une refactorisation pour vérifier que le comportement reste stable.

12. Nommage Visual Basic : conventions simples pour débiter

Le cours ne cherche pas à imposer un guide de style exhaustif. Il faut donner quelques habitudes constantes et faciles à appliquer.

Élément	Convention pédagogique	Exemple
Variable locale	camelCase	prixTotal
Constante	Nom explicite ; convention d'équipe cohérente	TauxTVA
Module / classe	PascalCase	Program, Contact
Méthode / procédure	PascalCase	AfficherContact
Booléen	Nom formulé comme un état	estMajeur, conversionOk

12.1 Atelier de renommage

```
Dim a As String = "Nora"
Dim b As String = "0612345678"
Dim c As Boolean = True
```

[MINI-DÉFI] Renommer a, b, c sans changer les valeurs. Demander ensuite à un autre étudiant de lire le programme.

12.2 Principe de décision

Un bon nom permet souvent de comprendre la donnée sans lire toute la ligne. Mais un nom très long n'est pas automatiquement meilleur. Chercher un nom suffisamment précis dans le contexte.

13. 15h30-16h15 - Fil rouge : première fiche Contact en console

13.1 Le besoin

Construire une première version très simple de l'application de gestion de contacts. Il n'y a pas encore de classe Contact ni de tableau de contacts ; ces notions arriveront plus tard. Aujourd'hui, un contact est représenté par quelques variables locales.

[QUESTION] « Quelles données minimales devons-nous mémoriser pour afficher une fiche contact ? »

Conserver trois champs : nom, téléphone, email. Éviter d'ajouter trop de règles, car les conditions sont prévues pour J2.

13.2 Pseudo-code collectif

```
DÉBUT
Demander le nom
Lire le nom
Demander le téléphone
Lire le téléphone
Demander l'email
Lire l'email
Afficher une fiche propre
FIN
```

13.3 Construction progressive

Étape 1 - nom uniquement :

```
Console.Write("Nom : ")
Dim nom As String = Console.ReadLine()
Console.WriteLine("Contact : " & nom)
```

Étape 2 - téléphone :

```
Console.Write("Téléphone : ")
Dim telephone As String = Console.ReadLine()
```

[QUESTION] « Pourquoi téléphone est-il un String et non un Integer ? »

[RÉPONSE ATTENDUE] On ne fait pas de calcul avec un numéro de téléphone ; il peut contenir des zéros initiaux, espaces ou symboles.

Étape 3 - email et affichage structuré :

```
Console.Write("Email : ")
Dim email As String = Console.ReadLine()

Console.WriteLine()
Console.WriteLine("--- FICHE CONTACT ---")
```

```

Console.WriteLine("Nom      : " & nom)
Console.WriteLine("Téléphone : " & telephone)
Console.WriteLine("Email    : " & email)

```

13.4 Variation autonome

Ajouter la ville ou l'année de naissance. Si l'année de naissance est choisie, demander quel type utiliser et pourquoi.

14. Version complète - Contact Console J1

Option Strict On

Module Program

```

Sub Main(args As String())
    Console.WriteLine("=== GESTION DE CONTACTS - J1 ===")
    Console.WriteLine()

    Console.Write("Nom : ")
    Dim nom As String = Console.ReadLine()

    Console.Write("Téléphone : ")
    Dim telephone As String = Console.ReadLine()

    Console.Write("Email : ")
    Dim email As String = Console.ReadLine()

    Console.Write("Année de naissance : ")
    Dim saisieAnnee As String = Console.ReadLine()
    Dim anneeNaissance As Integer
    Dim anneeValide As Boolean = Integer.TryParse(saisieAnnee, anneeNaissance)

    Console.WriteLine()
    Console.WriteLine("--- FICHE CONTACT ---")
    Console.WriteLine("Nom          : " & nom)
    Console.WriteLine("Téléphone    : " & telephone)
    Console.WriteLine("Email        : " & email)
    Console.WriteLine("Année naissance : " & anneeNaissance)
    Console.WriteLine("Conversion année : " & anneeValide)

End Sub
End Module

```

14.1 Ce qu'il faut faire expliquer par les étudiants

14. Pourquoi nom, téléphone et email sont des String.
15. Pourquoi l'année nécessite une conversion.
16. Ce que contient anneeValide.
17. Ce qui se passe si l'utilisateur saisit « abc » pour l'année.
18. Pourquoi le programme peut être amélioré demain avec une condition.

[NE PAS ANTICIPER] Ne pas corriger la saisie invalide avec une boucle ou un If complet pendant ce chapitre. Noter le besoin : il servira de pont vers J2.

15. 16h15-17h00 - TP 1 : Calculateur console

15.1 Énoncé étudiant

Créer une application console qui demande deux nombres puis affiche leur somme, différence, produit et division. Le programme doit utiliser des noms explicites et produire une sortie lisible.

Contraintes minimales

- Projet Console App Visual Basic.
- Deux saisies utilisateur.
- Conversion en Double ou Decimal.
- Au moins quatre calculs.
- Affichage avec libellés explicites.
- Code correctement indenté et variables correctement nommées.

Bonus raisonnables

- Afficher aussi la moyenne des deux nombres.
- Afficher le reste d'une division entière si les données choisies s'y prêtent.
- Utiliser TryParse pour au moins une saisie et afficher si la conversion a réussi.

15.2 Stratégie d'aide progressive

Niveau	Indice
Indice 1	Commencez par écrire le pseudo-code Entrées → Traitements → Sorties.
Indice 2	Console.ReadLine renvoie du texte.
Indice 3	Utilisez Double.Parse ou Double.TryParse.
Indice 4	Stockez chaque résultat dans une variable nommée.
Indice 5	Testez d'abord seulement l'addition, puis ajoutez une opération à la fois.

[FORMATEUR] Ne pas distribuer immédiatement la correction. Circuler, demander « qu'avez-vous déjà vérifié ? », puis donner l'indice le plus faible suffisant.

16. Correction live du TP : reconstruire à partir d'un projet vide

16.1 Étape 1 - pseudo-code

```
DÉBUT
Lire nombre1
Lire nombre2
Calculer somme
Calculer différence
Calculer produit
Calculer quotient
Afficher les résultats
FIN
```

16.2 Étape 2 - saisies

```
Console.WriteLine("Premier nombre : ")
Dim nombre1 As Double = Double.Parse(Console.ReadLine())

Console.WriteLine("Deuxième nombre : ")
Dim nombre2 As Double = Double.Parse(Console.ReadLine())
```

[QUESTION] « Quel risque contient déjà cette version ? »

[RÉPONSE ATTENDUE] Une saisie non numérique provoque une erreur d'exécution avec Parse.

16.3 Étape 3 - calculs

```
Dim somme As Double = nombre1 + nombre2
Dim difference As Double = nombre1 - nombre2
Dim produit As Double = nombre1 * nombre2
Dim quotient As Double = nombre1 / nombre2
```

16.4 Étape 4 - affichage

```
Console.WriteLine("Somme : " & somme)
Console.WriteLine("Différence : " & difference)
Console.WriteLine("Produit : " & produit)
Console.WriteLine("Quotient : " & quotient)
```

16.5 Variation 1 - moyenne

```
Dim moyenne As Double = (nombre1 + nombre2) / 2
```

16.6 Variation 2 - valeurs différentes

Relancer avec 10 et 4, puis avec -2 et 3. Faire prédire avant chaque lancement. Éviter encore la gestion complète de la division par zéro : noter le besoin pour J2.

17. Correction complète exécutable du calculateur

```
Option Strict On
```

```
Module Program
```

```
Sub Main(args As String())
```

```
    Console.WriteLine("=== CALCULATEUR J1 ===")
```

```
    Console.WriteLine()
```

```
    Console.Write("Premier nombre : ")
```

```
    Dim nombre1 As Double = Double.Parse(Console.ReadLine())
```

```
    Console.Write("Deuxième nombre : ")
```

```
    Dim nombre2 As Double = Double.Parse(Console.ReadLine())
```

```
    Dim somme As Double = nombre1 + nombre2
```

```
    Dim difference As Double = nombre1 - nombre2
```

```
    Dim produit As Double = nombre1 * nombre2
```

```
    Dim quotient As Double = nombre1 / nombre2
```

```
    Dim moyenne As Double = (nombre1 + nombre2) / 2
```

```
    Console.WriteLine()
```

```
    Console.WriteLine("--- RÉSULTATS ---")
```

```
    Console.WriteLine("Somme      : " & somme)
```

```
    Console.WriteLine("Différence : " & difference)
```

```
    Console.WriteLine("Produit   : " & produit)
```

```
    Console.WriteLine("Quotient  : " & quotient)
```

```
    Console.WriteLine("Moyenne   : " & moyenne)
```

```
End Sub
```

```
End Module
```

17.1 Variante pédagogique avec TryParse

```
Console.Write("Premier nombre : ")
```

```

Dim saisie1 As String = Console.ReadLine()
Dim nombre1 As Double
Dim ok1 As Boolean = Double.TryParse(saisie1, nombre1)

Console.WriteLine("Conversion réussie : " & ok1)
Console.WriteLine("Valeur obtenue : " & nombre1)

```

Cette variante ne traite pas encore le cas d'échec. Elle rend seulement visible le résultat de TryParse. Le traitement conditionnel sera naturel en J2.

18. Banque d'erreurs volontaires pour le J1

Ces erreurs permettent de transformer les problèmes réels en occasions d'apprentissage. Ne pas toutes utiliser si le rythme du groupe est lent.

Erreur injectée	Ce qu'elle permet de travailler	Question à poser
Parenthèse manquante	Compilation	« Quel indice donne le compilateur ? »
WritLine au lieu de WriteLine	Nom de membre / IntelliSense	« Est-ce une erreur métier ou de syntaxe ? »
Integer.Parse("abc")	Erreur d'exécution	« Pourquoi la compilation avait-elle réussi ? »
Dim age As Integer = 20.5	Type/conversion	« Quelle information perdrait-on ? »
Téléphone en Integer	Choix de type	« Fait-on des calculs avec ce nombre ? »
2 + 3 * 4	Priorité	« Quelle opération est évaluée d'abord ? »
Nom de variable x	Lisibilité	« Que représente x dans 5 minutes ? »
prixHT * tauxTVA au lieu de prixHT*(1+tauxTVA)	Erreur logique	« Le code tourne ; comment détecter que le résultat est faux ? »

18.1 Méthode de diagnostic à installer dès J1

19. Lire le message exact.
20. Repérer la ligne signalée.
21. Identifier ce qui était attendu.
22. Modifier une seule chose.
23. Relancer.
24. Vérifier le résultat avec une valeur simple connue.

[RÈGLE] Éviter le « je change plusieurs choses au hasard ». Le diagnostic est une démarche contrôlée.

19. Banque de questions de prédiction

Utiliser ces fragments à l'oral, au tableau ou directement dans Visual Studio. L'intérêt n'est pas le piège : c'est de forcer la construction d'un modèle mental.

Fragment	Résultat attendu
<code>Console.WriteLine(5 + 2)</code>	7
<code>Console.WriteLine("5" & "2")</code>	52
<code>Console.WriteLine(2 + 3 * 4)</code>	14
<code>Console.WriteLine((2 + 3) * 4)</code>	20
<code>Console.WriteLine(10 \ 3)</code>	3
<code>Console.WriteLine(10 Mod 3)</code>	1
<code>Console.WriteLine(5 > 2)</code>	True
<code>Console.WriteLine(5 = 2)</code>	False
<code>Console.WriteLine(True And False)</code>	False
<code>Console.WriteLine(True Or False)</code>	True
<code>Console.WriteLine(Not True)</code>	False

19.1 Questions conceptuelles courtes

- ReadLine renvoie-t-il directement un Integer ? Pourquoi ?
- Une variable peut-elle changer de valeur pendant le programme ?
- À quoi sert un type ?
- Quelle différence entre compilation et exécution ?
- Pourquoi un numéro de téléphone est-il souvent un String ?
- Que produit une comparaison ?
- And et Or répondent-ils au même besoin ?
- Pourquoi exécuter après chaque petite refactorisation ?

20. Exercices guidés supplémentaires

20.1 Exercice A - Convertisseur Celsius → Fahrenheit

Formule : $F = C \times 9 / 5 + 32$. Faire d'abord écrire l'algorithme, puis le code.

```

Console.Write("Température en °C : ")
Dim celsius As Double = Double.Parse(Console.ReadLine())
Dim fahrenheit As Double = celsius * 9 / 5 + 32

```

```
Console.WriteLine("Température en °F : " & fahrenheit)
```

20.2 Exercice B - Rectangle

Demander largeur et hauteur, puis afficher aire et périmètre.

```
Dim aire As Double = largeur * hauteur
```

```
Dim perimetre As Double = 2 * (largeur + hauteur)
```

20.3 Exercice C - Durée en minutes

À partir d'un nombre entier de minutes, calculer heures et minutes restantes.

```
Dim heures As Integer = totalMinutes \ 60
```

```
Dim minutesRestantes As Integer = totalMinutes Mod 60
```

20.4 Exercice D - Comparaisons

Créer deux âges et afficher le résultat de plusieurs comparaisons sans utiliser If.

```
Console.WriteLine(age1 > age2)
```

```
Console.WriteLine(age1 = age2)
```

```
Console.WriteLine(age1 <> age2)
```

21. Corrigés expliqués des exercices supplémentaires

21.1 Corrigé A - température

Le choix Double est cohérent car une température peut contenir des décimales. L'expression applique les règles de priorité usuelles ; les multiplications et divisions sont évaluées avant l'addition.

```
Option Strict On
```

```
Module Program
```

```
Sub Main(args As String())
```

```
    Console.Write("Température en °C : ")
```

```
    Dim celsius As Double = Double.Parse(Console.ReadLine())
```

```
    Dim fahrenheit As Double = celsius * 9 / 5 + 32
```

```
    Console.WriteLine(celsius & " °C = " & fahrenheit & " °F")
```

```
End Sub
```

```
End Module
```

21.2 Corrigé B - rectangle

```
Dim largeur As Double = 5
```

```
Dim hauteur As Double = 3
```

```
Dim aire As Double = largeur * hauteur
```

```
Dim perimetre As Double = 2 * (largeur + hauteur)
```

```
Console.WriteLine("Aire : " & aire)
```

```
Console.WriteLine("Périmètre : " & perimetre)
```

21.3 Corrigé C - minutes

```
Dim totalMinutes As Integer = 135
Dim heures As Integer = totalMinutes \ 60
Dim minutesRestantes As Integer = totalMinutes Mod 60
Console.WriteLine(heures & " h " & minutesRestantes & " min")
```

[EXPLICATION] 135 \ 60 vaut 2 car la division entière garde le quotient entier. 135 Mod 60 vaut 15 car il reste 15 après deux groupes de 60.

22. Ce qu'il faut écrire ou dessiner au tableau

22.1 Schéma 1 - Entrée / Traitement / Sortie

ENTRÉE	TRAITEMENT	SORTIE
texte tapé	conversion	message
nombre	calcul	résultat
	comparaison	True / False

22.2 Schéma 2 - De la source à l'exécution

```
CODE SOURCE (.vb)
↓
COMPILATION
↓
PROGRAMME CONSTRUIT
↓
EXÉCUTION
↓
SORTIE / COMPORTEMENT
```

22.3 Schéma 3 - Une variable

```
nom : age
┌───────────┐
type | Integer |
    | valeur = 25 |
└───────────┘
```

22.4 Schéma 4 - ReadLine et conversion

```
CLAVIER → "25" (String) → Integer.Parse / TryParse → 25 (Integer)
```

22.5 Schéma 5 - Expression booléenne

```
age >= 18 → True / False
↓
```

sera utilisée demain par If pour décider quoi faire

23. Questions fréquentes des débutants

« Pourquoi faut-il préciser As Integer ? »

Parce que le type décrit la nature de la donnée et aide le compilateur à vérifier les opérations. Même lorsque l'inférence est possible, expliciter certains types au début facilite l'apprentissage.

« Pourquoi ReadLine ne sait-il pas que j'ai tapé un nombre ? »

La console lit une suite de caractères. C'est votre programme qui décide comment interpréter cette suite : nombre, date, identifiant, téléphone, etc.

« Pourquoi Visual Basic utilise End Sub ? »

La syntaxe de Visual Basic utilise des mots-clés explicites pour marquer la fin de certaines structures. Cela rend la structure du code lisible sans accolades.

« Parse ou TryParse ? »

Parse est direct quand la donnée est supposée valide. TryParse est adapté aux saisies utilisateur parce qu'il permet de savoir si la conversion a réussi sans provoquer une exception pour une saisie incorrecte.

« Integer ou Double ? »

Integer pour les quantités entières ; Double quand des décimales générales sont nécessaires. Pour les montants financiers, Decimal est souvent un choix plus adapté.

« Pourquoi ne pas apprendre If tout de suite ? »

La progression sépare volontairement la construction d'une expression booléenne et la décision. Ainsi, demain, If aura un sens : il utilisera une question vraie/fausse déjà comprise.

24. Synthèse de fin de journée

24.1 Les 10 idées à faire reformuler

25. Un programme exécute des instructions.
26. Un algorithme décrit la solution indépendamment du langage.
27. Visual Basic est la syntaxe utilisée pour traduire cette solution.
28. La compilation vérifie et construit le programme avant exécution.
29. Une variable possède un nom, un type et une valeur.
30. ReadLine lit du texte.
31. Parse/TryParse servent à convertir.
32. Une expression arithmétique produit un nombre ; une comparaison produit un Boolean.
33. And, Or et Not combinent des booléens.
34. Un code lisible facilite la compréhension et la maintenance.

24.2 Quiz oral de sortie

Question	Réponse attendue
Quel type pour un prénom ?	String
Quel type pour un âge entier ?	Integer
Que renvoie ReadLine ?	String
Quel opérateur assemble clairement du texte ?	&
10 Mod 4 ?	2
5 > 3 ?	True
True And False ?	False
Pourquoi TryParse ?	Tester une conversion sans exception de format
Compilation ou exécution : parenthèse manquante ?	Compilation
Code qui tourne mais calcule faux ?	Erreur logique

[TRANSITION VERS J2] Conserver le programme Contact J1 et le calculateur. J2 partira précisément des besoins non traités aujourd'hui : que faire si une conversion échoue ? comment choisir une action ? comment répéter ? comment gérer plusieurs contacts ?

25. Fiche de référence rapide Visual Basic - J1

Déclarer

```
Dim age As Integer = 25
Dim prix As Decimal = 19.99D
Dim nom As String = "Nora"
Dim actif As Boolean = True
Const TauxTVA As Decimal = 0.2D
```

Lire / écrire

```
Console.Write("Nom : ")
Dim nom As String = Console.ReadLine()
Console.WriteLine("Bonjour " & nom)
```

Convertir

```
Dim age As Integer = Integer.Parse(Console.ReadLine())
Dim age2 As Integer
Dim ok As Boolean = Integer.TryParse(Console.ReadLine(), age2)
```

Calculer

+ - * / \ Mod ^

Comparer

= <> < > <= >=

Logique

And Or Not

Checklist avant de considérer un exercice terminé

- Le programme compile.
- Je l'ai exécuté avec au moins deux jeux de données.
- Les noms de variables expliquent leur rôle.
- Je sais dire quel type possède chaque variable.
- Je sais expliquer chaque conversion.
- Je peux prédire le résultat d'une expression avant de lancer.
- La sortie est lisible.

26. Annexes - scénarios de live coding prêts à l'emploi

Scénario A - 8 minutes : texte versus calcul

35. Taper `Console.WriteLine("5 + 2")`.
36. Faire prédire.
37. Exécuter : `5 + 2`.
38. Remplacer par `Console.WriteLine(5 + 2)`.
39. Faire prédire.
40. Exécuter : `7`.
41. Faire formuler : guillemets = texte ; sans guillemets = expression numérique.

Scénario B - 10 minutes : conversion

42. Lire une saisie dans un `String`.
43. Afficher la saisie.
44. Essayer de la traiter comme un nombre.
45. Introduire `Parse`.
46. Saisir `abc` et observer l'exception.
47. Remplacer par `TryParse`.
48. Afficher le booléen de réussite.

Scénario C - 10 minutes : erreur de compilation

49. Faire fonctionner une ligne `WriteLine`.
50. Supprimer une parenthèse.
51. Compiler.

- 52. Faire lire le message.
- 53. Corriger une seule chose.
- 54. Recompiler.
- 55. Faire verbaliser la méthode de diagnostic.

Scénario D - 12 minutes : refactorisation

- 56. Partir du code x/y/z.
- 57. Exécuter et noter le résultat.
- 58. Renommer x.
- 59. Relancer.
- 60. Renommer y puis z.
- 61. Introduire Const.
- 62. Améliorer la sortie.
- 63. Relancer et vérifier que le résultat n'a pas changé.

27. Checklist formateur - préparation et animation J1

- Créer ou tester les huit projets de démonstration avant le cours.
- Vérifier la version de .NET et la création de projets VB sur les postes.
- Préparer 3 valeurs de test simples pour chaque démonstration.
- Prévoir au moins 4 erreurs volontaires, mais ne pas toutes injecter si le groupe ralentit.
- Faire prédire avant les exécutions importantes.
- Laisser réellement quelques secondes de silence après une question.
- Ne pas corriger un étudiant avant qu'il ait expliqué son raisonnement.
- Repartir d'un projet vide lors de la correction finale du TP.
- Ne pas anticiper If, boucles ou classes au-delà d'une annonce de transition.
- Sauvegarder la version Contact J1 pour démarrer J2.

27.1 Critères d'observation pendant le TP

Critère	Signal positif	Aide si difficulté
Décomposition	L'étudiant écrit d'abord les étapes.	Revenir à Entrée/Traitement/Sortie.
Types	Choix cohérent entre texte/nombre.	Demander : « va-t-on calculer avec cette donnée ? »
Conversion	La saisie est convertie avant calcul.	Rappeler que ReadLine renvoie String.
Opérateurs	Résultats cohérents.	Faire tester avec 10 et 2.
Lisibilité	Noms explicites, indentation.	Faire relire le code par un binôme.
Diagnostic	Lit le message avant de modifier.	Demander : « quel est le message exact ? »

[FIN J1] Conserver une trace des erreurs fréquentes observées. Elles alimenteront la réactivation de J2.

28. Résumé ultra-court à relire avant J2

J1 a installé la chaîne complète : besoin → pseudo-code → code Visual Basic → compilation → exécution → observation. Les étudiants ont manipulé les données avec des variables typées, découvert que ReadLine fournit du texte, pratiqué Parse/TryParse, calculé avec des opérateurs, produit des booléens par comparaison, puis amélioré la lisibilité par refactorisation. Le fil rouge contient désormais une première fiche Contact console. Le TP calculateur sert de preuve qu'ils savent assembler ces notions dans un programme simple.

[PROCHAINE ÉTAPE] J2 ajoutera la décision (If/Select Case), la répétition (boucles), plusieurs données (tableaux) et le découpage en Sub / Function.

SUPPORT FORMATEUR ENRICHI

Visual Basic .NET - Jour 2

Structures de contrôle, tableaux, procédures et fonctions

Formation 3 jours / 21 heures - J2 : 7 heures - orientation live coding

Public	Débutants en programmation
Prérequis	Acquis de J1 : variables, types, conversions, opérateurs, expressions booléennes
Fil rouge	Mini-gestion de contacts en console
Approche	Micro-cycles : question → prédiction → code → exécution → observation → variation → mini-défi
Environnement	Visual Studio - Console App Visual Basic .NET
Objectif du jour	Rendre un programme capable de décider, répéter, mémoriser plusieurs données et se découper en traitements réutilisables.

Principe pédagogique : le code n'est pas montré comme une solution finie. Il est construit, exécuté, cassé, diagnostiqué et amélioré avec le groupe.

1. Positionnement du J2 dans la progression

Le J1 a posé les briques élémentaires : un programme lit des données, les stocke dans des variables typées, réalise des opérations et produit un résultat. Le J2 ajoute quatre capacités décisives : choisir, répéter, mémoriser plusieurs valeurs et découper le programme. Ces quatre capacités permettent de passer d'un programme linéaire à une petite application interactive.

[FIL CONDUCTEUR] À chaque notion, repartir d'un besoin du fil rouge Contacts : valider une saisie, afficher un menu, rechercher un contact, parcourir plusieurs contacts, extraire un traitement dans une Sub ou une Function.

Objectifs pédagogiques observables

- Choisir entre If...Then, If...Then...Else, ElseIf et Select Case à partir d'une règle métier simple.
- Construire et expliquer une boucle While, Do While, Do Until ou For...Next, notamment sa condition d'arrêt.
- Déclarer un tableau à une dimension, le parcourir et calculer un résultat à partir de ses éléments.
- Comprendre l'idée d'une matrice 2D et distinguer tableau, matrice et structures de données plus évoluées sans les approfondir.
- Extraire un traitement dans une Sub puis dans une Function.
- Expliquer la différence entre paramètre, argument, valeur de retour, ByVal et ByRef.
- Assembler ces notions dans une mini-gestion de contacts en console.

Rythme live coding conseillé

Étape	Durée typique	Rôle pédagogique
Question / problème	1-3 min	Faire apparaître le besoin avant la syntaxe.
Prédiction	1-2 min	Forcer un raisonnement avant l'exécution.
Construction du code	5-12 min	Taper progressivement, expliquer chaque ajout.
Exécution / observation	1-3 min	Lire le résultat plutôt que "croire" le code.
Erreur ou variation	3-8 min	Tester la solidité du modèle mental.
Mini-défi	5-15 min	Faire produire une modification autonome.
Correction / synthèse	3-8 min	Nommer la règle générale et relier au fil rouge.

2. Réactivation J1 - remettre les fondations en mémoire

La réactivation doit rester courte et active. Le but n'est pas de refaire J1, mais de vérifier que les prérequis indispensables aux conditions et aux boucles sont disponibles : variables, types, conversion et expressions booléennes.

2.1 Lecture de code : que va afficher ce programme ?

```
Module Program
    Sub Main()
        Dim age As Integer = 20
        Dim hasCard As Boolean = True
        Dim allowed As Boolean = age >= 18 And hasCard

        Console.WriteLine(allowed)
    End Sub
End Module
```

[PRÉDICTION] Demander une réponse avant d'exécuter : True ou False ? Puis demander quels sous-résultats composent l'expression.

Faire verbaliser : `age >= 18` produit un Boolean. `hasCard` est déjà un Boolean. L'opérateur `And` combine les deux. Cette lecture prépare directement les conditions : un `If` ne fait qu'utiliser une expression booléenne pour choisir un chemin.

2.2 Mini-défi de conversion

```
Console.Write("Âge : ")
Dim raw As String = Console.ReadLine()
Dim age As Integer
Dim ok As Boolean = Integer.TryParse(raw, age)
```

[QUESTION] Que contient `ok` ? Que contient `age` lorsque la conversion réussit ? Pourquoi cette forme est-elle préférable à `Parse` lorsque la saisie vient d'un utilisateur ?

2.3 Point de contrôle

- Une comparaison produit un Boolean.
- Une expression logique combine des Boolean.
- `TryParse` permet de savoir si la conversion a réussi sans déclencher une erreur de format.
- Ces trois éléments vont être réutilisés immédiatement dans `If` et dans les conditions d'arrêt des boucles.

3. Choisir un chemin : If...Then

3.1 Le besoin avant la syntaxe

Un programme linéaire exécute toutes ses instructions dans l'ordre. Mais une règle métier dit souvent : “si cette condition est vraie, faire quelque chose”. Le mot important est “si”. Une structure conditionnelle permet de rendre l'exécution dépendante d'une expression booléenne.

[TABLEAU] Dessiner deux boîtes : CONDITION ? puis deux flèches “True” et “False”. Insister : la condition ne contient pas une action, elle produit un Boolean.

3.2 Premier live coding : contrôler l'accès

```
Console.WriteLine("Âge : ")
Dim age As Integer = Integer.Parse(Console.ReadLine())

If age >= 18 Then
    Console.WriteLine("Accès autorisé")
End If
```

[EXÉCUTION] Tester 20 puis 15. Pour 15, rien ne s'affiche après la saisie : ce silence est pédagogiquement utile, car il montre qu'un If simple n'a pas forcément de branche False.

Lecture ligne par ligne

- If démarre la décision.
- age >= 18 est l'expression booléenne.
- Then introduit le bloc exécuté si la condition vaut True.
- End If ferme explicitement la structure.

3.3 Erreurs volontaires utiles

```
If age >= 18
    Console.WriteLine("Accès autorisé")
End If
```

[DIAGNOSTIC] Faire lire le message de compilation et faire repérer l'absence de Then avant de corriger.

```
If age = 18 Then
    Console.WriteLine("Majeur")
End If
```

En Visual Basic, = est utilisé pour la comparaison d'égalité dans une expression. Comparer cette syntaxe à C#/Java où == est utilisé, uniquement pour éviter une confusion de syntaxe.

3.4 Mini-défis

1. Afficher “Température négative” si une température est inférieure à 0.
2. Afficher “Mot de passe assez long” si la longueur saisie est au moins 8.
3. Dans le fil rouge, afficher “Nom manquant” si le nom saisi est vide après Trim().

4. Deux chemins : If...Then...Else

Lorsqu'il faut choisir exactement entre deux comportements, le If simple devient insuffisant. Else représente le chemin suivi lorsque la condition est False.

4.1 Live coding : majeur ou mineur

```
If age >= 18 Then
    Console.WriteLine("Majeur")
Else
    Console.WriteLine("Mineur")
End If
```

[PRÉDICTION] Faire prédire les sorties pour 18, 17 et 0. Le cas 18 permet de rappeler l'importance de >= par rapport à >.

4.2 Penser en partition de cas

Un bon If...Else découpe l'ensemble des entrées en deux parties qui couvrent tous les cas. Pour age >= 18, Else signifie implicitement age < 18. Il est inutile de réécrire la condition contraire.

```
If age >= 18 Then
    Console.WriteLine("Majeur")
ElseIf age < 18 Then ' inutile ici
    Console.WriteLine("Mineur")
End If
```

[CORRECTION] Montrer la version If...Else et demander : “Quel cas pourrait arriver dans Else qui ne soit pas age < 18 ?” Aucun pour un Integer.

4.3 Fil rouge : validation d'un nom

```
Console.Write("Nom : ")
Dim name As String = Console.ReadLine().Trim()

If name = "" Then
    Console.WriteLine("Erreur : le nom est obligatoire.")
Else
    Console.WriteLine("Contact accepté : " & name)
End If
```

Variation

Demander aux étudiants de modifier le programme pour afficher un message spécifique lorsque le nom fait moins de 2 caractères. Cette variante prépare ElseIf.

5. Plusieurs cas : Elself

5.1 Pourquoi ne pas empiler des If indépendants ?

```
If score >= 90 Then Console.WriteLine("Excellent")
If score >= 70 Then Console.WriteLine("Bien")
If score >= 50 Then Console.WriteLine("Passable")
```

[PRÉDICTION] Pour score = 95, combien de lignes s'affichent ? Trois. Le problème n'est pas la syntaxe : c'est que les trois décisions sont indépendantes.

5.2 Construire une chaîne exclusive

```
If score >= 90 Then
    Console.WriteLine("Excellent")
ElseIf score >= 70 Then
    Console.WriteLine("Bien")
ElseIf score >= 50 Then
    Console.WriteLine("Passable")
Else
    Console.WriteLine("Insuffisant")
End If
```

Une chaîne ElseIf s'arrête à la première condition vraie. L'ordre est donc une partie de la logique. Ici, on part du seuil le plus élevé vers le plus faible.

5.3 Erreur d'ordre volontaire

```
If score >= 50 Then
    Console.WriteLine("Passable")
ElseIf score >= 70 Then
    Console.WriteLine("Bien")
ElseIf score >= 90 Then
    Console.WriteLine("Excellent")
End If
```

[DIAGNOSTIC] Avec 95, la première condition est déjà vraie. Les suivantes ne sont jamais testées. Faire corriger l'ordre sans donner immédiatement la réponse.

5.4 Cas métier : niveau de priorité d'un contact

On peut simuler une règle simple selon un nombre de jours depuis le dernier échange. Le but n'est pas de créer une vraie règle CRM, mais de travailler la hiérarchie des conditions.

```
If daysSinceLastContact >= 90 Then
    Console.WriteLine("Priorité élevée")
ElseIf daysSinceLastContact >= 30 Then
    Console.WriteLine("À relancer")
Else
    Console.WriteLine("Contact récent")
End If
```

6. Select Case - choisir parmi des valeurs discrètes

Select Case devient lisible lorsqu'une même valeur est comparée à plusieurs cas distincts. Il évite une longue chaîne de ElseIf lorsque la décision est essentiellement "quelle valeur a cette variable ?".

6.1 Live coding : menu utilisateur

```

Console.WriteLine("1 - Ajouter")
Console.WriteLine("2 - Afficher")
Console.WriteLine("3 - Rechercher")
Console.WriteLine("0 - Quitter")
Console.Write("Choix : ")

Dim choice As String = Console.ReadLine()

Select Case choice
    Case "1"
        Console.WriteLine("Ajout")
    Case "2"
        Console.WriteLine("Affichage")
    Case "3"
        Console.WriteLine("Recherche")
    Case "0"
        Console.WriteLine("Au revoir")
    Case Else
        Console.WriteLine("Choix invalide")
End Select

```

[QUESTION] Pourquoi lire choice comme String ici ? Réponse attendue : les options sont des symboles de menu ; aucune opération arithmétique n'est nécessaire. Cela évite une conversion inutile.

6.2 If ou Select Case ?

Situation	Choix naturel	Pourquoi
Âge >= 18	If	Condition basée sur une comparaison / plage.
Score avec seuils 90,70,50	If / ElseIf	Les cas sont des intervalles ordonnés.
Menu 1,2,3,0	Select Case	Une variable, plusieurs valeurs discrètes.
Nom vide ou non	If...Else	Deux cas simples.
Commande A/V/R/Q	Select Case	Lisibilité d'un menu.

6.3 Mini-défi

Ajouter une option 4 - Compter les contacts. Puis modifier le Case Else pour afficher le choix réellement reçu.

7. Boucles : répéter sans copier-coller

Une boucle répond à un autre problème : exécuter plusieurs fois un bloc d'instructions. Le danger principal n'est pas la syntaxe, mais la maîtrise de la condition d'arrêt. Une boucle correcte doit avoir une raison claire de continuer et une manière claire de finir.

7.1 Modèle mental commun

Question	Exemple
Que répète-t-on ?	Afficher une ligne, demander un choix, traiter un contact.
Quand commence-t-on ?	Immédiatement ou seulement si une condition est vraie.
Quand s'arrête-t-on ?	Après N répétitions, lorsque l'utilisateur quitte, lorsque la donnée est trouvée.
Qu'est-ce qui change à chaque tour ?	Compteur, saisie, index, total cumulé.
[TABLEAU] Dessiner : TEST → BLOC → MISE À JOUR → retour au TEST. Pour une boucle basée sur compteur, entourer la mise à jour : si elle disparaît, la boucle peut devenir infinie.	

8. While - répéter tant qu'une condition est vraie

8.1 Première boucle contrôlée par compteur

```
Dim i As Integer = 1

While i <= 5
    Console.WriteLine(i)
    i += 1
End While
```

[PRÉDICTION] Avant exécution : quelles valeurs s'affichent ? Quelle est la valeur de i après la boucle ?

Décomposer la boucle

- Initialisation : i vaut 1 avant la boucle.
- Test : i <= 5 décide si un nouveau tour démarre.
- Corps : Console.WriteLine(i).
- Mise à jour : i += 1 rapproche la boucle de son arrêt.

8.2 Erreur volontaire : boucle infinie

```
Dim i As Integer = 1
While i <= 5
    Console.WriteLine(i)
End While
```

[ERREUR VOLONTAIRE] Exécuter brièvement puis arrêter le programme. Demander : "La condition peut-elle devenir False ?" Non, car i ne change jamais.

8.3 While pour valider une saisie

```
Dim age As Integer
Dim ok As Boolean = False

While Not ok
    Console.Write("Âge : ")
    ok = Integer.TryParse(Console.ReadLine(), age)

    If Not ok Then
        Console.WriteLine("Saisie invalide.")
    End If
End While

Console.WriteLine("Âge accepté : " & age)
```

Cette version relie directement J1 (TryParse, Boolean) à J2 (répétition). La boucle continue tant que la saisie n'est pas valide.

9. Do While et Do Until - exprimer la condition autrement

9.1 Do While

```
Dim choice As String = ""
```

```
Do While choice <> "0"
```

```
    Console.Write("Choix (0 pour quitter) : ")
```

```
    choice = Console.ReadLine()
```

```
    Console.WriteLine("Vous avez choisi : " & choice)
```

```
Loop
```

Do While exprime : “répéter tant que la condition est vraie”. Ici, tant que choice n’est pas "0".

9.2 Do Until

```
Dim choice As String = ""
```

```
Do Until choice = "0"
```

```
    Console.Write("Choix (0 pour quitter) : ")
```

```
    choice = Console.ReadLine()
```

```
Loop
```

Do Until exprime la même intention depuis l’autre côté : “répéter jusqu’à ce que la condition devienne vraie”. Les deux formulations peuvent être logiquement équivalentes.

9.3 Choisir la formulation la plus lisible

Besoin	Formulation intuitive
Continuer tant que le mot de passe est invalide	While Not valid / Do While Not valid
Continuer jusqu’à un choix 0	Do Until choice = "0"
Continuer tant qu’il reste des tentatives	While attempts > 0
Continuer jusqu’à ce qu’un contact soit trouvé	Do Until found

[RÈGLE] Choisir la forme qui traduit le plus clairement la phrase métier. La lisibilité prime sur la variété syntaxique.

10. For...Next - répéter un nombre connu de fois

10.1 Live coding : table de multiplication

```

Console.Write("Nombre : ")
Dim n As Integer = Integer.Parse(Console.ReadLine())

For i As Integer = 1 To 10
    Console.WriteLine(n & " x " & i & " = " & (n * i))
Next

```

[PRÉDICTION] Combien de tours ? Quelle est la première valeur de i ? La dernière ? Pourquoi For est-il plus naturel ici qu'un While ?

10.2 Accumulateur

```

Dim total As Integer = 0

For i As Integer = 1 To 5
    Console.WriteLine("Valeur " & i & " : ")
    Dim value As Integer = Integer.Parse(Console.ReadLine())
    total += value
Next

```

```

Console.WriteLine("Total = " & total)

```

Un accumulateur conserve un résultat partiel d'un tour à l'autre. Il doit être initialisé avant la boucle, puis mis à jour dans la boucle.

10.3 Erreur fréquente : initialiser dans la boucle

```

For i As Integer = 1 To 5
    Dim total As Integer = 0
    total += i
Next

```

[DIAGNOSTIC] Le total est recréé à chaque tour. Demander où placer l'initialisation pour conserver la valeur entre les tours.

11. Boucles imbriquées et matrices : comprendre sans surcharger

Une boucle peut contenir une autre boucle. Cette idée est utile pour parcourir une structure à deux dimensions. Le déroulé J2 prévoit une introduction aux matrices, pas un approfondissement algorithmique.

11.1 Exemple visuel 2 x 3

```
Dim grid(1, 2) As Integer
```

```
grid(0, 0) = 10
```

```
grid(0, 1) = 20
```

```
grid(0, 2) = 30
```

```
grid(1, 0) = 40
```

```
grid(1, 1) = 50
```

```
grid(1, 2) = 60
```

[TABLEAU] Dessiner un tableau à 2 lignes et 3 colonnes. Inscrire les indices (0,0), (0,1), etc. Insister : l'indice démarre à 0.

11.2 Parcours imbriqué

```
For row As Integer = 0 To 1
```

```
    For col As Integer = 0 To 2
```

```
        Console.Write(grid(row, col) & " ")
```

```
    Next
```

```
    Console.WriteLine()
```

```
Next
```

[PRÉDICTION] Combien de fois l'instruction Write est-elle exécutée ? Réponse : $2 \times 3 = 6$.

Ne pas introduire de structures plus complexes ici. Les arbres et autres organisations sont uniquement mentionnés comme aperçu de structures évoluées, conformément au déroulé.

12. Tableaux à une dimension - mémoriser plusieurs valeurs du même type

12.1 Pourquoi un tableau ?

Avec une seule note, une variable suffit. Avec 20 notes, créer note1, note2, ... note20 devient fragile. Un tableau regroupe plusieurs valeurs du même type sous un seul nom, chaque valeur étant repérée par un index.

```
Dim notes() As Integer = {12, 15, 9, 18, 14}
```

[QUESTION] Quel est l'index de la première note ? 0. Quel est l'index de la dernière ? 4. Combien y a-t-il d'éléments ? 5.

12.2 Parcours avec For

```
For i As Integer = 0 To notes.Length - 1
    Console.WriteLine("Index " & i & " : " & notes(i))
Next
```

Length donne le nombre d'éléments. Comme les indices commencent à 0, le dernier index est Length - 1.

12.3 Erreur volontaire : sortir du tableau

```
For i As Integer = 0 To notes.Length
    Console.WriteLine(notes(i))
Next
```

[ERREUR VOLONTAIRE] Exécuter pour provoquer l'accès à un index inexistant. Faire observer que la compilation réussit : l'erreur apparaît pendant l'exécution.

12.4 Calculer moyenne, min et max

```
Dim total As Integer = 0
Dim minimum As Integer = notes(0)
Dim maximum As Integer = notes(0)

For i As Integer = 0 To notes.Length - 1
    total += notes(i)

    If notes(i) < minimum Then
        minimum = notes(i)
    End If

    If notes(i) > maximum Then
        maximum = notes(i)
    End If
Next

Dim average As Double = total / notes.Length
Console.WriteLine("Moyenne : " & average)
Console.WriteLine("Min : " & minimum)
```

```
Console.WriteLine("Max : " & maximum)
```

13. Rechercher dans un tableau

13.1 Recherche simple

```
Dim names() As String = {"Alice", "Bruno", "Chloé", "David"}
Console.Write("Nom recherché : ")
Dim target As String = Console.ReadLine().Trim()
Dim found As Boolean = False
```

```
For i As Integer = 0 To names.Length - 1
    If names(i) = target Then
        found = True
    End If
Next
```

```
If found Then
    Console.WriteLine("Contact trouvé")
Else
    Console.WriteLine("Contact absent")
End If
```

Cette démonstration combine tableau, boucle, condition et drapeau booléen. Le mot “drapeau” peut être expliqué comme une variable qui mémorise un état : trouvé / non trouvé.

13.2 Faire apparaître une limite

Même si found devient True, la boucle continue jusqu’au bout. Pour ce niveau, on peut conserver cette version claire. L’objectif est de comprendre le mécanisme de recherche, pas d’optimiser prématurément.

13.3 Variation insensible à la casse

```
If names(i).ToLower() = target.ToLower() Then
    found = True
End If
```

[DISCUSSION] Présenter cette variation comme une amélioration pratique, sans transformer la séance en cours complet sur les chaînes.

14. Fil rouge - de plusieurs variables à plusieurs contacts

À ce stade, le groupe sait représenter plusieurs valeurs homogènes, mais pas encore un objet Contact : la programmation objet est prévue pour J3. On reste donc volontairement sur des tableaux simples afin de respecter la progression.

14.1 Version pédagogique : noms et téléphones en tableaux parallèles

```
Dim names(4) As String
Dim phones(4) As String
Dim count As Integer = 0
```

Deux tableaux parallèles signifient que names(0) et phones(0) décrivent le même contact, names(1) et phones(1) le suivant, etc. Cette solution est volontairement simple et prépare le besoin d'une classe Contact au J3.

[QUESTION] Quel problème apparaît si l'on ajoute ensuite email, ville et date de naissance ?

Réponse attendue : on multiplie les tableaux parallèles et le code devient plus fragile. Garder cette question en mémoire pour J3.

14.2 Ajouter un contact

```
If count < names.Length Then
    Console.WriteLine("Nom : ")
    names(count) = Console.ReadLine().Trim()

    Console.WriteLine("Téléphone : ")
    phones(count) = Console.ReadLine().Trim()

    count += 1
    Console.WriteLine("Contact ajouté.")
Else
    Console.WriteLine("Capacité maximale atteinte.")
End If
```

14.3 Afficher les contacts existants

```
For i As Integer = 0 To count - 1
    Console.WriteLine((i + 1) & " - " & names(i) & " - " & phones(i))
Next
```

Pourquoi boucler jusqu'à count - 1 et non jusqu'à names.Length - 1 ? Parce que le tableau peut contenir des emplacements encore inutilisés.

15. Procédures Sub - arrêter de tout mettre dans Main

15.1 Le symptôme : Main devient trop long

Après plusieurs fonctions de menu, le programme principal mélange affichage, saisie, règles et traitement. Le problème n'est pas seulement esthétique : il devient plus difficile de comprendre, modifier et tester chaque partie.

15.2 Première extraction

```
Sub ShowMenu()
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("3 - Rechercher")
    Console.WriteLine("0 - Quitter")
End Sub

Sub Main()
    ShowMenu()
End Sub
```

[QUESTION] Qu'est-ce que ShowMenu reçoit ? Rien. Qu'est-ce qu'elle renvoie ? Rien. Elle réalise une action : afficher le menu.

15.3 Une Sub avec paramètres

```
Sub PrintContact(name As String, phone As String)
    Console.WriteLine(name & " - " & phone)
End Sub

PrintContact("Alice", "06 00 00 00 00")
```

Dans la déclaration, name et phone sont des paramètres. Lors de l'appel, "Alice" et le numéro sont des arguments.

15.4 Règle de conception simple

[RÈGLE] Une Sub doit avoir un rôle clair que l'on peut nommer par un verbe : ShowMenu, PrintContact, AddContact. Si le nom devient "DoEverything", le découpage est probablement insuffisant.

16. Refactorisation live : du programme monolithique aux Sub

Cette séquence doit être réalisée dans l'éditeur en partant d'un code qui fonctionne. L'objectif est de montrer qu'une refactorisation améliore l'organisation sans changer le comportement visible.

16.1 Version initiale

```
Sub Main()
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("0 - Quitter")
    Console.Write("Choix : ")
    Dim choice As String = Console.ReadLine()

    If choice = "1" Then
        Console.Write("Nom : ")
        Dim name As String = Console.ReadLine()
        Console.WriteLine("Ajout : " & name)
    ElseIf choice = "2" Then
        Console.WriteLine("Affichage des contacts")
    End If
End Sub
```

16.2 Étape 1 : extraire l'affichage du menu

```
Sub ShowMenu()
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("0 - Quitter")
End Sub
```

16.3 Étape 2 : extraire la lecture du choix

```
Function ReadChoice() As String
    Console.Write("Choix : ")
    Return Console.ReadLine()
End Function
```

[TRANSITION] Cette extraction fait apparaître naturellement la différence Sub / Function : ReadChoice doit fournir une valeur au code appelant.

17. Function - calculer et retourner une valeur

17.1 Différence essentielle avec Sub

Sub	Function
Réalise principalement une action.	Produit une valeur utilisable par l'appelant.
Pas de valeur de retour.	Type de retour déclaré avec As ...
Appel : ShowMenu()	Appel dans une expression : result = Add(a,b)
Exemple : afficher un contact.	Exemple : vérifier si une saisie est valide.

17.2 Première Function

```
Function Add(a As Integer, b As Integer) As Integer
    Return a + b
End Function
Dim result As Integer = Add(4, 7)
Console.WriteLine(result)
```

[PRÉDICTION] Quelle valeur est copiée dans result ? Quel est le type du résultat ?

17.3 Function métier : validation d'un nom

```
Function IsValidName(name As String) As Boolean
    Return name.Trim().Length >= 2
End Function
If IsValidName(name) Then
    Console.WriteLine("Nom valide")
Else
    Console.WriteLine("Nom invalide")
End If
```

Cette forme est pédagogique car elle donne un nom à une règle : le code appelant se lit presque comme une phrase.

18. Paramètres, arguments et portée locale

18.1 Paramètre vs argument

Le paramètre existe dans la définition d'une procédure ou fonction. L'argument est la valeur ou l'expression fournie au moment de l'appel.

```
Function Multiply(a As Integer, b As Integer) As Integer
    Return a * b
End Function
```

```
Dim total As Integer = Multiply(3, 5)
```

Élément	Dans l'exemple
Paramètres	a et b
Arguments	3 et 5
Valeur de retour	15
Variable qui reçoit le retour	total

18.2 Portée locale

```
Sub Example()
    Dim message As String = "Bonjour"
    Console.WriteLine(message)
End Sub
```

message est locale à Example. Une variable locale sert seulement dans le bloc où elle est déclarée. Cette limitation évite qu'une partie du programme modifie accidentellement des données qui ne la concernent pas.

18.3 Erreur volontaire de portée

```
Sub Example()
    Dim message As String = "Bonjour"
End Sub
```

```
Sub Main()
    Console.WriteLine(message)
End Sub
```

[DIAGNOSTIC] Faire compiler et lire l'erreur : message n'existe pas dans le contexte de Main. Puis expliquer que la variable locale disparaît de la portée accessible en sortant de Example.

19. ByVal - protéger l'appelant des réaffectations locales

En Visual Basic, les paramètres sont ByVal par défaut. Pour des types simples comme Integer, la fonction reçoit une valeur distincte pour le paramètre. Réaffecter ce paramètre ne change pas directement la variable passée par l'appelant.

19.1 Démonstration

```
Sub Increase(ByVal number As Integer)
    number += 1
    Console.WriteLine("Dans Increase : " & number)
End Sub
```

```
Sub Main()
    Dim x As Integer = 10
    Increase(x)
    Console.WriteLine("Dans Main : " & x)
End Sub
```

[PRÉDICTION] Avant d'exécuter : x va-t-il valoir 10 ou 11 après l'appel ?

Résultat attendu : 11 est affiché dans Increase, mais x reste 10 dans Main. Ce contraste doit être observé à l'exécution.

19.2 Pourquoi cela aide

ByVal limite certains effets de bord : une procédure ne peut pas réaffecter directement la variable simple de l'appelant via son paramètre. Cela rend les traitements plus faciles à raisonner.

20. ByRef - permettre une modification visible par l'appelant

ByRef signifie que le paramètre donne accès à la variable de l'appelant de manière à ce qu'une réaffectation puisse être visible après l'appel. Cette puissance doit être utilisée consciemment, car elle crée un effet de bord.

20.1 Démonstration comparative

```
Sub IncreaseByRef(ByRef number As Integer)
    number += 1
End Sub
```

```
Sub Main()
    Dim x As Integer = 10
    IncreaseByRef(x)
    Console.WriteLine(x)
End Sub
```

[PRÉDICTION] Quelle valeur affiche Main maintenant ? 11. Faire comparer directement avec la version ByVal.

20.2 ByRef pour récupérer plusieurs résultats : démonstration pédagogique

```
Sub Divide(a As Integer, b As Integer, ByRef quotient As Integer, ByRef remainder As Integer)
    quotient = a \ b
    remainder = a Mod b
End Sub
```

Cette démonstration montre un cas possible, mais ne doit pas devenir une recommandation systématique. Le point pédagogique est de comprendre que ByRef modifie une variable fournie par l'appelant.

20.3 Règle de décision

[RÈGLE] Commencer par ByVal. Utiliser ByRef uniquement lorsque la modification de la variable de l'appelant fait explicitement partie du contrat de la procédure.

21. Effets de bord : rendre les fonctions prévisibles

Un effet de bord est une modification observable de l'état du programme autre que la valeur de retour. Écrire dans la console, modifier une variable ByRef ou changer une donnée partagée sont des exemples d'effets de bord. À ce niveau, l'objectif est surtout de savoir les repérer.

21.1 Fonction facile à raisonner

```
Function CalculateTotal(price As Decimal, quantity As Integer) As Decimal
    Return price * quantity
End Function
```

À entrées identiques, le résultat est directement calculable. La fonction ne demande aucune saisie et n'affiche rien.

21.2 Fonction plus difficile à réutiliser

```
Function CalculateTotal() As Decimal
    Console.Write("Prix : ")
    Dim price As Decimal = Decimal.Parse(Console.ReadLine())
    Console.Write("Quantité : ")
    Dim quantity As Integer = Integer.Parse(Console.ReadLine())
    Console.WriteLine("Calcul en cours...")
    Return price * quantity
End Function
```

Cette version mélange lecture utilisateur, calcul et affichage. Le calcul devient difficile à réutiliser et à tester isolément. On prépare ici l'idée de fonctions courtes et ciblées, sans anticiper le cours de tests de J3.

22. Construire un menu répétitif - assemblage des notions

22.1 Première version

```
Dim choice As String

Do
    ShowMenu()
    choice = ReadChoice()

    Select Case choice
        Case "1"
            Console.WriteLine("Ajouter")
        Case "2"
            Console.WriteLine("Afficher")
        Case "3"
            Console.WriteLine("Rechercher")
        Case "0"
            Console.WriteLine("Au revoir")
        Case Else
            Console.WriteLine("Choix invalide")
    End Select
Loop Until choice = "0"
```

[LECTURE] Faire suivre le flux à voix haute : afficher → lire → choisir → traiter → revenir au début, jusqu'à 0.

22.2 Erreur volontaire

```
Loop Until choice <> "0"
```

[DIAGNOSTIC] Le programme s'arrête après presque n'importe quel choix. Faire reformuler la phrase : "je veux boucler jusqu'à ce que choice soit égal à 0".

23. Construire les fonctions du fil rouge

23.1 Afficher le menu

```
Sub ShowMenu()
    Console.WriteLine()
    Console.WriteLine("=== CONTACTS ===")
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("3 - Rechercher")
    Console.WriteLine("0 - Quitter")
End Sub
```

23.2 Lire le choix

```
Function ReadChoice() As String
    Console.Write("Choix : ")
    Return Console.ReadLine().Trim()
End Function
```

23.3 Valider un nom

```
Function IsValidName(name As String) As Boolean
    Return name.Trim().Length >= 2
End Function
```

23.4 Afficher un contact

```
Sub PrintContact(name As String, phone As String)
    Console.WriteLine(name & " - " & phone)
End Sub
```

À chaque extraction, demander : “Action ou valeur ?” Si le bloc réalise une action, une Sub est souvent naturelle. S’il calcule ou décide quelque chose qui doit être réutilisé par l’appelant, une Function est souvent plus appropriée.

24. TP 2 - Mini-gestion de contacts : énoncé progressif

Le TP doit réutiliser les notions de J2 sans introduire la classe Contact, réservée au J3. Le stockage repose donc sur deux tableaux parallèles, solution volontairement transitoire.

Niveau 1 - menu et boucle

4. Afficher les options Ajouter, Afficher, Rechercher, Quitter.
5. Répéter le menu jusqu'au choix 0.
6. Gérer un choix invalide avec Case Else.

Niveau 2 - ajout

7. Créer deux tableaux names et phones de taille 10.
8. Créer count pour connaître le nombre de contacts réellement stockés.
9. Ajouter un contact à l'index count puis incrémenter count.
10. Refuser l'ajout si le tableau est plein.

Niveau 3 - affichage

11. Parcourir uniquement les indices 0 à count - 1.
12. Afficher un numéro humain i + 1 ainsi que le nom et le téléphone.
13. Afficher "Aucun contact" si count = 0.

Niveau 4 - recherche

14. Demander un nom.
15. Parcourir les contacts existants.
16. Mémoriser si le contact a été trouvé.
17. Afficher le numéro correspondant ou un message d'absence.

Niveau 5 - modularisation

18. Extraire ShowMenu.
19. Extraire ReadChoice.
20. Extraire PrintContact.
21. Extraire IsValidName.
22. Identifier au moins une autre partie pouvant devenir une Sub ou une Function.

[AIDE PROGRESSIVE] Ne pas donner la correction complète immédiatement. Donner d'abord une question, puis un indice de structure, puis un fragment, et seulement ensuite la correction.

25. TP 2 - correction progressive : squelette

```

Module Program
    Sub Main()
        Dim names(9) As String
        Dim phones(9) As String
        Dim count As Integer = 0
        Dim choice As String

        Do
            ShowMenu()
            choice = ReadChoice()

            Select Case choice
                Case "1"
                    ' Ajouter
                Case "2"
                    ' Afficher
                Case "3"
                    ' Rechercher
                Case "0"
                    Console.WriteLine("Au revoir")
                Case Else
                    Console.WriteLine("Choix invalide")
            End Select
            Loop Until choice = "0"
        End Sub
    End Module

```

Le squelette rend visible l'architecture générale sans résoudre les détails. Faire compléter une option à la fois et exécuter après chaque étape.

25.1 Correction de l'ajout

```

If count >= names.Length Then
    Console.WriteLine("Capacité maximale atteinte.")
Else
    Console.Write("Nom : ")
    Dim name As String = Console.ReadLine().Trim()

    If IsValidName(name) Then
        Console.Write("Téléphone : ")
        Dim phone As String = Console.ReadLine().Trim()

        names(count) = name
        phones(count) = phone
        count += 1
        Console.WriteLine("Contact ajouté.")
    Else
        Console.WriteLine("Nom invalide.")
    End If
End If

```

```
End If
End If
```

26. TP 2 - correction progressive : affichage et recherche

26.1 Afficher

```
If count = 0 Then
    Console.WriteLine("Aucun contact.")
Else
    For i As Integer = 0 To count - 1
        PrintContact(names(i), phones(i))
    Next
End If
```

26.2 Rechercher

```
Console.Write("Nom recherché : ")
Dim target As String = Console.ReadLine().Trim()
Dim found As Boolean = False

For i As Integer = 0 To count - 1
    If names(i).ToLower() = target.ToLower() Then
        PrintContact(names(i), phones(i))
        found = True
    End If
Next

If Not found Then
    Console.WriteLine("Contact non trouvé.")
End If
```

[POINT DE CONTRÔLE] Demander à un étudiant de montrer où sont utilisées : une condition, une boucle, un tableau, une Sub, une Function et un Boolean de contrôle.

27. TP 2 - version complète exécutable

Module Program

Sub Main()

Dim names(9) As String

Dim phones(9) As String

Dim count As Integer = 0

Dim choice As String

Do

ShowMenu()

choice = ReadChoice()

Select Case choice

Case "1"

If count >= names.Length Then

Console.WriteLine("Capacité maximale atteinte.")

Else

Console.Write("Nom : ")

Dim name As String = Console.ReadLine().Trim()

If IsValidName(name) Then

Console.Write("Téléphone : ")

Dim phone As String = Console.ReadLine().Trim()

names(count) = name

phones(count) = phone

count += 1

Console.WriteLine("Contact ajouté.")

Else

Console.WriteLine("Nom invalide.")

End If

End If

Case "2"

If count = 0 Then

Console.WriteLine("Aucun contact.")

Else

For i As Integer = 0 To count - 1

PrintContact(names(i), phones(i))

Next

End If

Case "3"

Console.Write("Nom recherché : ")

Dim target As String = Console.ReadLine().Trim()

Dim found As Boolean = False

For i As Integer = 0 To count - 1

```

        If names(i).ToLower() = target.ToLower() Then
            PrintContact(names(i), phones(i))
            found = True
        End If
    Next

    If Not found Then
        Console.WriteLine("Contact non trouvé.")
    End If

    Case "0"
        Console.WriteLine("Au revoir")

    Case Else
        Console.WriteLine("Choix invalide")
    End Select
    Loop Until choice = "0"
End Sub

Sub ShowMenu()
    Console.WriteLine()
    Console.WriteLine("=== CONTACTS ===")
    Console.WriteLine("1 - Ajouter")
    Console.WriteLine("2 - Afficher")
    Console.WriteLine("3 - Rechercher")
    Console.WriteLine("0 - Quitter")
End Sub

Function ReadChoice() As String
    Console.Write("Choix : ")
    Return Console.ReadLine().Trim()
End Function

Function IsValidName(name As String) As Boolean
    Return name.Trim().Length >= 2
End Function

Sub PrintContact(name As String, phone As String)
    Console.WriteLine(name & " - " & phone)
End Sub
End Module

```

28. Lecture critique de la version complète

La correction n'est pas la fin du TP. Elle sert de support à une lecture architecturale simple : quelles responsabilités sont déjà séparées ? Qu'est-ce qui reste encore mélangé ?

Ce qui est déjà mieux qu'un Main monolithique

- Le menu est isolé dans ShowMenu.
- La lecture du choix est isolée dans ReadChoice.
- La validation du nom est nommée par IsValidName.
- L'affichage d'un contact est centralisé dans PrintContact.
- Le flux général reste visible dans Main.

Limites volontairement conservées

- Les données d'un contact sont réparties dans deux tableaux parallèles.
- Les traitements Ajouter/Afficher/Rechercher restent encore longs dans Main.
- Le stockage disparaît à la fermeture du programme.
- Il n'existe pas encore de classe Contact ni de base de données.

[TRANSITION J3] Ces limites créent précisément les besoins du J3 : regrouper les données dans un objet Contact, organiser davantage le code et connecter une base de données.

29. Atelier de débogage J2 - erreurs à injecter

Même si le débogueur complet est traité au J3, le J2 peut déjà entraîner le diagnostic par observation des erreurs et des sorties. Les bugs suivants sont adaptés aux notions du jour.

Bug injecté	Symptôme	Question de diagnostic
Oublier count += 1	Les contacts écrasent toujours le premier emplacement.	Quelle variable représente le prochain index libre ?
Boucler To count au lieu de To count - 1	Accès à un emplacement non utilisé ou comportement incohérent.	Quel est le dernier index réellement rempli ?
Loop Until choice <> "0"	Le menu s'arrête trop tôt.	La phrase logique correspond-elle au besoin ?
Initialiser found = True	Toute recherche semble réussie.	Quelle doit être la valeur avant d'avoir trouvé quoi que ce soit ?
Utiliser If indépendants pour les options	Plusieurs traitements peuvent être déclenchés selon la logique choisie.	Les choix sont-ils exclusifs ?
Passer un calcul en Sub alors qu'une valeur est nécessaire	Le code appelant ne récupère pas le résultat.	Action ou valeur ?

30. Exercices d'entraînement complémentaires

Exercice A - tarif selon âge

Lire un âge et afficher Gratuit pour moins de 6 ans, Réduit pour 6 à 17 ans, Plein tarif à partir de 18 ans. Utiliser une chaîne If / ElseIf / Else.

Exercice B - menu de calcul

Créer un menu 1 Addition, 2 Soustraction, 3 Multiplication, 0 Quitter. Utiliser Select Case et une boucle de menu.

Exercice C - statistiques de notes

À partir d'un tableau de notes, calculer le total, la moyenne, le minimum et le maximum.

Exercice D - recherche

À partir d'un tableau de prénoms, demander un prénom et indiquer s'il est présent.

Exercice E - modularisation

Transformer un programme qui contient 40 lignes dans Main en plusieurs Sub et Function nommées de façon explicite.

Exercice F - ByVal / ByRef

Écrire deux procédures qui ajoutent 10 à un Integer : l'une ByVal, l'autre ByRef. Comparer les valeurs avant, pendant et après les appels.

31. Corrections des exercices complémentaires

Correction A

```
If age < 6 Then
    Console.WriteLine("Gratuit")
ElseIf age < 18 Then
    Console.WriteLine("Réduit")
Else
    Console.WriteLine("Plein tarif")
End If
```

Correction B - structure

```
Dim choice As String
Do
    Console.WriteLine("1 Addition")
    Console.WriteLine("2 Soustraction")
    Console.WriteLine("3 Multiplication")
    Console.WriteLine("0 Quitter")
    choice = Console.ReadLine()

    Select Case choice
        Case "1"
            Console.WriteLine("Addition")
        Case "2"
            Console.WriteLine("Soustraction")
        Case "3"
            Console.WriteLine("Multiplication")
        Case "0"
            Console.WriteLine("Fin")
        Case Else
            Console.WriteLine("Choix invalide")
    End Select
Loop Until choice = "0"
```

Correction C - idée centrale

Initialiser total à 0, minimum et maximum au premier élément, parcourir le tableau puis calculer $\text{average} = \text{total} / \text{notes.Length}$.

Correction D - idée centrale

Initialiser found à False, parcourir, passer found à True si l'élément correspond, puis afficher le résultat après la boucle.

Correction E - critères

- Une responsabilité claire par procédure.
- Un nom qui décrit l'intention.
- Éviter de dupliquer l'affichage du menu.
- Transformer en Fonction les traitements qui doivent fournir une valeur au code appelant.

Correction F - sortie attendue

La version ByVal ne change pas la variable de Main après l'appel. La version ByRef change la variable de Main.

32. Questions fréquentes et réponses pédagogiques

Pourquoi les indices commencent-ils à 0 ?

Pour le cours, retenir surtout la conséquence pratique : dans un tableau de Length éléments, les indices vont de 0 à Length - 1. Le détail historique ou bas niveau n'est pas nécessaire pour réussir les exercices.

Quelle différence entre While et For ?

For est naturel lorsque le nombre de répétitions ou la plage d'indices est connu. While est naturel lorsqu'on répète tant qu'une condition reste vraie sans connaître nécessairement le nombre de tours à l'avance.

Quelle différence entre Do While et While ?

Les deux peuvent exprimer "tant que". La forme Do permet aussi une écriture avec Until. L'important est la lisibilité de la condition et la compréhension du moment où elle est évaluée dans la forme utilisée.

Pourquoi une Function plutôt qu'une Sub avec ByRef ?

Lorsqu'un traitement produit naturellement une seule valeur, une Function rend généralement l'intention plus claire. ByRef est utile lorsque modifier explicitement une variable de l'appelant fait partie du contrat.

Pourquoi ne pas créer Contact comme classe aujourd'hui ?

Parce que la progression prévoit l'introduction à la programmation objet au J3. Les tableaux parallèles servent volontairement de solution transitoire qui fera apparaître le besoin de regrouper les données.

33. Checklist de qualité du code J2

Point de contrôle	Question à se poser
Conditions	Les cas sont-ils exclusifs ? L'ordre des ElseIf est-il correct ?
Boucles	La condition d'arrêt peut-elle réellement devenir vraie ?
Compteurs	L'initialisation et la mise à jour sont-elles au bon endroit ?
Tableaux	Les indices vont-ils de 0 à Length - 1 ?
Recherche	found démarre-t-il à False ?
Sub	La procédure réalise-t-elle une action clairement nommée ?
Function	La valeur de retour est-elle utile au code appelant ?
Paramètres	Les types correspondent-ils aux données attendues ?
ByRef	La modification de l'appelant est-elle intentionnelle ?
Main	Le flux général reste-t-il lisible ?

34. Synthèse conceptuelle du J2

Concept	Question résolue	Syntaxe / repère
Condition	Quel chemin exécuter ?	If / ElseIf / Else
Sélection discrète	Quelle option parmi plusieurs ?	Select Case
Boucle conditionnelle	Combien de fois répéter si le nombre n'est pas connu ?	While / Do While / Do Until
Boucle comptée	Répéter sur une plage connue	For...Next
Tableau	Comment regrouper plusieurs valeurs du même type ?	Dim values() As Type
Sub	Comment nommer et réutiliser une action ?	Sub ... End Sub
Function	Comment nommer un calcul / une décision qui produit une valeur ?	Function ... As Type / Return
ByVal	Comment passer une valeur sans réaffecter la variable simple de l'appelant ?	ByVal
ByRef	Comment autoriser la modification visible chez l'appelant ?	ByRef

[SYNTHÈSE] J1 : stocker et calculer. J2 : décider, répéter, regrouper et découper. J3 : regrouper les données dans des objets, persister en base, diagnostiquer et tester.

35. Quiz de fin de journée - version formateur

1. Quelle structure convient naturellement à un menu 1/2/3/0 ?

Réponse : Select Case.

2. Que vaut le dernier index d'un tableau de Length 5 ?

Réponse : 4.

3. Quel risque apparaît si la variable de contrôle d'un While ne change jamais ?

Réponse : Une boucle infinie.

4. Quelle différence essentielle entre Sub et Function ?

Réponse : Function renvoie une valeur ; Sub réalise une action sans valeur de retour.

5. Que fait ByRef ?

Réponse : Il permet qu'une réaffectation du paramètre modifie la variable de l'appelant.

6. Pourquoi utiliser count avec un tableau partiellement rempli ?

Réponse : Pour savoir combien d'éléments sont réellement utilisés.

7. Pourquoi found démarre-t-il généralement à False ?

Réponse : Parce qu'aucun élément n'a encore été trouvé au début de la recherche.

8. Pourquoi ElseIf doit-il parfois être ordonné du seuil le plus élevé au plus faible ?

Réponse : Parce que la première condition vraie arrête la chaîne.

9. Quel est le rôle de Return ?

Réponse : Fournir la valeur produite par une Function.

10. Quel besoin du J3 est créé par les tableaux parallèles ?

Réponse : Regrouper les données liées dans un objet Contact.

36. Déroulement live coding conseillé - conducteur synthétique J2

Créneau	Focus	Dynamique live coding
09:00-09:20	Réactivation J1	Lecture de code, prédictions, correction orale.
09:20-10:20	Conditions	Construire If puis ElseIf, casser l'ordre, basculer vers Select Case.
10:35-11:45	Boucles	While → erreur infinie → Do → For → accumulateur.
11:45-12:30	Tableaux	Indices, parcours, erreur hors limites, moyenne/recherche.
13:30-14:20	Sub	Refactoriser un Main trop long, une extraction à la fois.
14:20-15:20	Function / paramètres	Action vs valeur, Return, ByVal/ByRef avec prédictions.
15:35-16:45	TP Contacts	Menu répétitif, tableaux parallèles, ajout/affichage/recherche, modularisation.
16:45-17:00	Consolidation	Faire reformuler 3 règles de conception et annoncer le besoin objet du J3.

37. Annexes - fragments de code prêts pour les démonstrations

A. If minimal

```
If temperature < 0 Then
    Console.WriteLine("Gel")
End If
```

B. Elself

```
If value > 0 Then
    Console.WriteLine("Positif")
ElseIf value < 0 Then
    Console.WriteLine("Négatif")
Else
    Console.WriteLine("Zéro")
End If
```

C. Select Case

```
Select Case command
    Case "A"
        Console.WriteLine("Ajouter")
    Case "Q"
        Console.WriteLine("Quitter")
    Case Else
        Console.WriteLine("Inconnu")
End Select
```

D. While

```
Dim i As Integer = 0
While i < 3
    Console.WriteLine(i)
    i += 1
End While
```

E. For

```
For i As Integer = 0 To values.Length - 1
    Console.WriteLine(values(i))
Next
```

F. Function

```
Function IsAdult(age As Integer) As Boolean
    Return age >= 18
End Function
```

38. Annexes - idées d'erreurs volontaires classées par notion

Notion	Erreur volontaire	Ce que l'on veut faire comprendre
If	Inverser > et >=	Les frontières font partie de la règle métier.
ElseIf	Mettre le seuil le plus faible en premier	La première condition vraie gagne.
While	Oublier la mise à jour	La condition d'arrêt doit pouvoir évoluer.
For / tableau	Utiliser Length comme dernier index	Dernier index = Length - 1.
Accumulateur	Réinitialiser total dans la boucle	L'état cumulé doit survivre entre les tours.
Recherche	Initialiser found à True	La valeur initiale doit représenter l'état réel avant la recherche.
Sub	Tout laisser dans Main	Le découpage réduit la complexité locale.
Function	Oublier Return	Une Function doit fournir sa valeur au code appelant.
ByVal/ByRef	Prédire la même sortie	Le mode de passage change l'effet sur la variable appelante.

39. Annexes - règles à faire formuler par les étudiants

23. Une condition doit être formulable comme une question dont la réponse est True ou False.
24. Une boucle doit avoir une condition d'arrêt compréhensible et atteignable.
25. Le dernier index d'un tableau est Length - 1.
26. Un accumulateur s'initialise avant la boucle.
27. Une recherche a besoin d'un état initial "pas encore trouvé".
28. Une Sub réalise une action ; une Function produit une valeur.
29. Un nom de procédure doit expliquer l'intention mieux que les détails de syntaxe.
30. ByRef ne doit pas être utilisé par habitude : la modification de l'appelant doit être volontaire.
31. On exécute souvent et on modifie une seule chose à la fois en live coding.
32. Un code qui fonctionne peut encore être amélioré par refactorisation sans changer son comportement.

40. Point de contrôle final J2

À la fin de la journée, l'étudiant doit pouvoir expliquer et montrer dans du code les capacités suivantes :

- Choisir une structure conditionnelle adaptée et justifier ce choix.
- Construire une boucle dont il peut expliquer l'initialisation, la condition et l'évolution.
- Parcourir un tableau sans dépasser ses indices.
- Utiliser une boucle et une condition pour rechercher une valeur.
- Extraire un bloc dans une Sub.
- Créer une Function avec paramètres et valeur de retour.
- Prédire la différence de comportement entre ByVal et ByRef sur un Integer.
- Lire le flux général de la mini-gestion de contacts et identifier les limites qui motivent l'introduction de la classe Contact au J3.

[ÉVALUATION J2] Capacité à sélectionner conditions/boucles adaptées, parcourir un tableau, découper un traitement et expliquer la différence entre Sub, Function, ByVal et ByRef.

Le J2 doit se terminer sur une application fonctionnelle, mais aussi sur une question ouverte : comment éviter de maintenir séparément le nom et le téléphone d'un même contact ? Cette question lance naturellement le J3 et la programmation objet.

SUPPORT FORMATEUR ENRICHI

Visual Basic .NET - Jour 3

Objet, données, interface, débogage et tests

Formation 3 jours / 21 heures - J3 : 7 heures - orientation live coding

Public	Débutants ayant suivi J1 et J2
Prérequis	Conditions, boucles, tableaux, Sub, Function, paramètres et bases de ByVal/ByRef
Fil rouge	Gestion de contacts : passage de la console vers objets, base SQLite, WinForms, débogage et tests
Approche	70-80 % éditeur / exécution / diagnostic ; 20-30 % schémas et synthèses
Environnement	Visual Studio, Visual Basic .NET, WinForms, Microsoft.Data.Sqlite, MSTest
Objectif du jour	Transformer le fil rouge en une petite application structurée, persistante, observable et testable.
Règle live coding	Question → prédiction → code → exécution → observation → erreur utile → diagnostic → variation → synthèse

Ce support est volontairement plus détaillé que ce qui sera dit mot à mot : il sert de réserve d'explications, de conducteur de démonstrations, de banque de bugs et de référence pendant les questions des étudiants.

1. Positionnement du J3 : passer d'un programme qui fonctionne à une petite application structurée

J1 a construit la capacité à lire, transformer et afficher des données. J2 a ajouté la décision, la répétition, les collections simples et le découpage en procédures et fonctions. J3 change de niveau : on ne cherche plus seulement à faire fonctionner un traitement, mais à organiser les responsabilités, conserver des données, déclencher un traitement depuis une interface graphique, comprendre une anomalie et sécuriser une règle avec un test.

[IDÉE CLÉ] Le J3 ne doit pas être présenté comme six mini-cours indépendants. Classe, base de données, WinForms, débogage et tests sont cinq manières d'améliorer le même fil rouge Contacts.

Objectifs observables

- Créer une classe Contact et instancier plusieurs objets.
- Expliquer ce qui appartient au Contact et ce qui appartient à l'interface ou à l'accès aux données.
- Exécuter une requête paramétrée et transformer les lignes lues en objets Contact.
- Relier un clic WinForms à une recherche puis à l'affichage.
- Distinguer erreur de compilation, erreur d'exécution et erreur de logique.
- Utiliser un breakpoint, Step Over/Into, Locals et Watch.
- Écrire et interpréter au moins un test MSTest selon Arrange-Act-Assert.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

2. Carte de la journée et stratégie live coding

Créneau	But	Production observable
09h00-09h20	Réactivation J2	Lecture d'un extrait + variation exécutée
09h20-10h30	Classe Contact	Contact.vb + instances
10h45-11h30	Organisation	Séparation interaction / logique / données
11h30-12h30	ADO.NET	Recherche paramétrée exécutée
13h30-14h35	WinForms	Formulaire de recherche fonctionnel
14h35-15h20	Erreurs	3 bugs classés et diagnostiqués
15h35-16h20	Débogueur	Bug localisé pas à pas
16h20-16h50	Tests	Un test rouge puis vert
16h50-17h00	Clôture	Trajet complet d'une recherche expliqué

[ANIMATION] Exécuter très souvent. Une démonstration de 20 minutes sans exécution intermédiaire n'est plus vraiment du live coding : elle devient une dictée de code.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

3. Réactivation J2 - lire avant d'écrire

Ouvrir le projet de mini-gestion de contacts de J2. Ne pas commencer par expliquer la POO. Commencer par demander aux étudiants ce qu'ils reconnaissent : menu, boucle principale, tableau, Sub d'affichage, Function de recherche. La première activité du J3 doit faire sentir que le nouveau contenu répond à des limites concrètes du code déjà produit.

```
Dim names() As String = {"Alice", "Bob", "Chloé"}
Dim phones() As String = {"0600000001", "0600000002", "0600000003"}

For i As Integer = 0 To names.Length - 1
    Console.WriteLine(names(i) & " - " & phones(i))
Next
```

[QUESTION] Qu'est-ce qui garantit que names(1) et phones(1) appartiennent à la même personne ?

Réponse attendue : uniquement notre convention d'index. Le langage ne représente pas encore explicitement le concept de contact.

[TRANSITION] « Aujourd'hui, nous allons apprendre à dire au programme qu'un nom, un téléphone et un email forment UNE même chose : un Contact. »

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

4. Du besoin à l'objet : faire émerger le concept Contact

Éviter de commencer par une définition abstraite de “classe”. Faire d’abord verbaliser le métier. Un contact possède des informations et on veut effectuer quelques actions le concernant. En programmation objet, on cherche précisément à regrouper dans une même structure les données qui décrivent une chose et, lorsque c’est pertinent, les comportements qui lui appartiennent.

Questions à poser

- Quelles informations définissent un contact dans notre application ?
- Qu’est-ce qui pourrait être calculé ou validé à partir de ces informations ?
- Quelles actions appartiennent vraiment au contact, et lesquelles appartiennent à l’écran ou à la base ?

[TABLEAU] Dessiner une boîte « Contact » et deux zones : DONNÉES (Name, Phone, Email) / COMPORTEMENTS (DisplayLabel, IsEmailValid par exemple). Ne parler de syntaxe VB qu’après ce dessin.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu’il n’autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L’objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l’instance utilisées dans l’exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l’interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

5. Classe et objet : deux mots à distinguer immédiatement

Une classe est un modèle de fabrication. Un objet est une instance concrète créée à partir de ce modèle. La classe Contact décrit ce qu'un contact sait contenir et faire ; alice et bob sont deux objets distincts construits selon ce même modèle.

Terme	Formulation simple	Exemple
Classe	Le modèle / le plan	Contact
Objet / instance	Une occurrence concrète créée à partir du modèle	alice, bob
Propriété	Une donnée exposée par l'objet	Name, Phone, Email
Méthode	Une action disponible sur l'objet	DisplayLabel()
Constructeur	Le code exécuté lors de la création	Sub New(...)

[ERREUR FRÉQUENTE] Dire « la classe Alice » alors qu'Alice est un objet. Corriger systématiquement ce vocabulaire au début : cela évite beaucoup de confusion ensuite.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

6. Première classe Contact - la version minimale

Dans Visual Studio, ajouter un fichier Contact.vb. Taper uniquement la structure minimale, compiler immédiatement, puis revenir au fichier Program.vb. Le but est de faire comprendre qu'une classe est d'abord un nouveau type que nous définissons nous-mêmes.

```
Public Class Contact
End Class
```

[PRÉDICTION] Le projet compile-t-il avec une classe vide ? Oui. Demander ensuite : peut-on déjà écrire Dim c As Contact ? Oui : le type existe.

```
Dim c As Contact
Console.WriteLine("Le type Contact existe.")
```

[POINT DE CONTRÔLE] Les étudiants doivent être capables de dire : « Integer est un type fourni par .NET ; Contact est un type que nous venons de définir. »

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

7. Ajouter des propriétés une par une

Ne pas coller les trois propriétés d'un bloc. Ajouter Name, compiler, instancier, affecter, lire. Ajouter ensuite Phone, puis Email. Cette lenteur apparente est pédagogique : elle rend visible le lien entre définition du type et possibilités disponibles dans l'éditeur.

```
Public Class Contact
    Public Property Name As String
End Class
Dim c As New Contact()
c.Name = "Alice"
Console.WriteLine(c.Name)
```

[QUESTION] Que représente le point dans c.Name ? Il signifie que l'on accède à un membre de l'objet référencé par c.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

8. Version Contact avec trois propriétés

```
Public Class Contact
    Public Property Name As String
    Public Property Phone As String
    Public Property Email As String
End Class
```

À ce stade, les propriétés sont publiques et modifiables librement. C'est volontaire : le cours vise une introduction intuitive. On expliquera l'idée d'encapsulation sans entrer dans des constructions avancées qui détourneraient l'attention de la structure globale.

[VARIATION] Créer deux objets avec des valeurs différentes. Modifier c1.Name et vérifier que c2.Name ne change pas. Faire formuler : « deux instances partagent la même structure, pas les mêmes données ».

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

9. Le constructeur Sub New : garantir une initialisation cohérente

Sans constructeur explicite, on peut créer un Contact puis oublier de lui donner un nom. Le constructeur permet de demander certaines informations dès la création. Introduire Sub New comme une étape de sécurisation du modèle, pas comme une syntaxe à mémoriser.

```
Public Class Contact
    Public Property Name As String
    Public Property Phone As String
    Public Property Email As String

    Public Sub New(name As String, phone As String, email As String)
        Me.Name = name
        Me.Phone = phone
        Me.Email = email
    End Sub
End Class
```

[À DIRE] Me désigne l'objet en cours de construction. Me.Name est la propriété ; name est le paramètre reçu.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

10. Instancier avec New - observer ce que change le constructeur

```
Dim alice As New Contact("Alice", "0600000001", "alice@example.com")
```

```
Dim bob As New Contact("Bob", "0600000002", "bob@example.com")
```

```
Console.WriteLine(alice.Name)
```

```
Console.WriteLine(bob.Email)
```

[ERREUR VOLONTAIRE] Tenter `Dim c As New Contact()` après avoir défini uniquement le constructeur à 3 paramètres. Lire le message de compilation. Faire diagnostiquer : la signature demandée n'existe plus.

[RÈGLE] `New` déclenche le constructeur correspondant aux arguments fournis.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à `Contact` et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

11. Méthode simple : donner un comportement au Contact

Ajouter une méthode qui produit un libellé d'affichage. Elle a du sens dans Contact car elle dépend directement de ses propriétés et ne connaît ni Console, ni WinForms, ni base de données.

```
Public Function DisplayLabel() As String
    Return Name & " - " & Phone & " - " & Email
End Function
Console.WriteLine(alice.DisplayLabel())
```

[QUESTION] Pourquoi préférer retourner une String plutôt que faire Console.WriteLine directement dans Contact ? Parce que le même Contact pourra ensuite être affiché dans la console, une ListBox, un DataGridView ou un test.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

12. Responsabilité : première règle d'organisation objet

Une responsabilité est une raison claire pour laquelle un morceau de code existe. Contact doit représenter un contact et éventuellement fournir de petits comportements liés à lui. Il ne doit pas ouvrir une fenêtre, lire le clavier ou gérer une connexion SQL. Ces responsabilités appartiennent à d'autres parties de l'application.

Code / action	Responsabilité naturelle
Stocker Name, Phone, Email	Contact
Construire un libellé	Contact
Lire une saisie clavier	Programme / interface
Afficher une MessageBox	Formulaire
Exécuter SELECT	Accès aux données
Décider si une saisie est vide	Logique de validation, éventuellement fonction dédiée

[POINT DE CONTRÔLE] Faire classer oralement cinq nouvelles actions par responsabilité. Le but est de faire émerger l'organisation avant de parler de couches ou d'architecture.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

13. Encapsulation intuitive - protéger un état cohérent

L'encapsulation ne doit pas être transformée en cours complet de conception. La définition utile ici : un objet doit éviter de laisser son état devenir incohérent. On peut commencer par centraliser une validation dans une méthode ou dans le constructeur.

```
Public Sub New(name As String, phone As String, email As String)
    If String.IsNullOrEmpty(name) Then
        Throw New ArgumentException("Le nom est obligatoire.")
    End If

    Me.Name = name.Trim()
    Me.Phone = phone.Trim()
    Me.Email = email.Trim()
End Sub
```

[VARIATION] Créer volontairement `New Contact("", "0600", "a@b.fr")` et observer l'exception. Ne pas approfondir encore Try/Catch : l'objectif est d'illustrer que l'objet peut refuser un état invalide.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à `Contact` et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

14. Plusieurs objets : remplacer les tableaux parallèles

Reprendre les tableaux `names()` et `phones()` du début et les remplacer par un tableau de `Contact`. La valeur pédagogique est forte : les données qui décrivent une même personne voyagent ensemble.

```
Dim contacts() As Contact = {
    New Contact("Alice", "0600000001", "alice@example.com"),
    New Contact("Bob", "0600000002", "bob@example.com"),
    New Contact("Chloé", "0600000003", "chloe@example.com")
}
```

```
For Each contact As Contact In contacts
    Console.WriteLine(contact.DisplayLabel())
Next
```

[QUESTION] Qu'est-ce qui a disparu ? Les index synchronisés entre plusieurs tableaux. Chaque élément possède désormais toutes les données du contact.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à `Contact` et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

15. Recherche sur des objets

```
Public Function FindByName(contacts() As Contact, search As String) As Contact
    For Each contact As Contact In contacts
        If contact.Name.ToLower() = search.ToLower() Then
            Return contact
        End If
    Next

    Return Nothing
End Function
```

Nothing signifie ici qu'aucun objet Contact n'a été trouvé. C'est une bonne occasion de rappeler qu'une Function peut retourner autre chose qu'un nombre ou un Boolean : elle peut retourner un objet.

[PRÉDICTION] Que se passe-t-il si on écrit result.Name alors que result vaut Nothing ? Une NullReferenceException apparaît à l'exécution.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

16. Tester Nothing avant d'utiliser l'objet

```
Dim result As Contact = FindByName(contacts, "Bob")
```

```
If result Is Nothing Then
```

```
    Console.WriteLine("Aucun contact trouvé.")
```

```
Else
```

```
    Console.WriteLine(result.DisplayLabel())
```

```
End If
```

[ERREUR VOLONTAIRE] Chercher "Zoe", supprimer le test Is Nothing et exécuter. Lire l'exception avant de corriger. Ce bug sera réutilisé pendant la séquence débogueur.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

17. Refactorisation live : interaction, logique, données

À ce stade, le programme console mélange probablement menu, saisie, recherche, affichage et stockage. Le but de la séquence 10h45-11h30 est de déplacer progressivement les responsabilités sans changer le comportement visible.

[TABLEAU] Dessiner trois zones : INTERACTION (Console / WinForms) → LOGIQUE (validation, recherche) → DONNÉES (tableau puis base). Ajouter Contact comme objet qui circule entre les zones.

Éviter des termes d'architecture trop lourds. On ne prétend pas enseigner MVC, Clean Architecture ou Repository en 45 minutes. On enseigne une intuition : chaque partie du programme doit avoir un rôle compréhensible.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

18. Première extraction : ne plus mélanger lecture et calcul

```
Public Function AskSearch() As String
    Console.WriteLine("Nom recherché : ")
    Return Console.ReadLine().Trim()
End Function
Dim search As String = AskSearch()
Dim result As Contact = FindByName(contacts, search)
```

[QUESTION] Qu'est-ce qui a changé pour l'utilisateur ? Rien. Qu'est-ce qui a changé pour le développeur ? Les responsabilités sont plus visibles et la recherche peut être appelée sans dépendre du clavier.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

19. Refactorisation sans peur : exécuter après chaque déplacement

Présenter une règle pratique : une refactorisation cherche à améliorer la structure sans changer le comportement attendu. Dans un cours live coding, cette règle se vérifie immédiatement : déplacer une fonction, compiler, relancer le même scénario. Si le résultat change, on a probablement modifié le comportement en plus de la structure.

Étape	Conduite formateur
Besoin	Le programme fonctionne mais devient long.
Prédiction	Que devrait afficher le même scénario après extraction ? Exactement la même chose.
Code / action	Extraire une fonction, compiler, relancer.
Observation	Comparer la sortie avant/après.
Variation / mini-défi	Extraire ensuite l'affichage dans une Sub, puis faire refaire une extraction par les étudiants.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

20. Transition vers les données persistantes : pourquoi une base ?

Le tableau en mémoire disparaît lorsque le programme s'arrête. Une base de données permet de conserver les contacts entre deux exécutions et de rechercher sans charger manuellement toutes les données. Ne pas commencer par ADO.NET : commencer par cette limite concrète du tableau.

[QUESTION] Si je ferme l'application maintenant, où sont mes contacts ? Ils n'existent plus, sauf s'ils étaient écrits en dur dans le code. Quel nouveau besoin apparaît ? La persistance.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

21. Vocabulaire base de données minimal

Terme	Image mentale	Dans Contacts
Table	Un ensemble de lignes du même type	Contacts
Ligne	Un enregistrement	Un contact
Colonne	Une information commune à toutes les lignes	Name, Phone, Email
Clé primaire	Identifiant unique	Id
Requête	Instruction envoyée au moteur	SELECT ... WHERE ...
Paramètre	Valeur fournie séparément du texte SQL	@name

[RÈGLE] Le but du J3 n'est pas d'enseigner SQL en profondeur. On utilise juste assez de SQL pour comprendre le trajet d'une recherche depuis l'application.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

22. Base de démonstration SQLite

Pour garder le live coding local et reproductible, on peut utiliser SQLite. Le moteur travaille dans un fichier .db et l'application l'ouvre via un fournisseur ADO.NET. Avec .NET moderne, le package Microsoft.Data.Sqlite fournit SQLiteConnection, SQLiteCommand et SQLiteDataReader.

```
dotnet add package Microsoft.Data.Sqlite
```

[FORMATEUR] Préparer le package avant la séance si le réseau est incertain. Le live coding doit porter sur la programmation, pas sur l'attente d'un téléchargement.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

23. Créer la table Contacts une fois

```
CREATE TABLE Contacts (
  Id INTEGER PRIMARY KEY AUTOINCREMENT,
  Name TEXT NOT NULL,
  Phone TEXT NOT NULL,
  Email TEXT NOT NULL
);
INSERT INTO Contacts(Name, Phone, Email) VALUES
('Alice', '0600000001', 'alice@example.com'),
('Bob', '0600000002', 'bob@example.com'),
('Chloé', '0600000003', 'chloe@example.com');
```

Cette préparation peut être faite à l'avance. Montrer toutefois la table avec un outil SQLite ou une extension Visual Studio afin que les étudiants voient les lignes réelles avant d'écrire le code d'accès.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

24. ADO.NET : le trajet conceptuel avant la syntaxe

[TABLEAU] Dessiner : APPLICATION → CONNECTION → COMMAND → DATABASE → READER → OBJETS CONTACT → AFFICHAGE.

Objet	Question à laquelle il répond
Connection	À quelle base voulons-nous parler ?
Command	Quelle requête voulons-nous exécuter ?
Parameter	Quelle valeur utilisateur est injectée dans la requête ?
Reader	Comment lire les lignes renvoyées ?
Contact	Comment représenter chaque ligne côté application ?

Présenter ADO.NET comme un ensemble d'objets .NET destinés à communiquer avec une source de données. SQLite n'est qu'un fournisseur concret utilisé pour la démonstration.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

25. Première connexion SQLite

```
Imports Microsoft.Data.Sqlite

Dim connectionString As String = "Data Source=contacts.db"

Using connection As New SqliteConnection(connectionString)
    connection.Open()
    Console.WriteLine("Connexion ouverte.")
End Using
```

Using garantit que la connexion sera libérée à la fin du bloc, même si le code évolue. Pour des débutants, retenir surtout : ouvrir le plus tard possible, fermer automatiquement le plus tôt possible.

[PRÉDICTION] Que se passe-t-il si le fichier n'existe pas ? SQLite peut le créer. Mais la table Contacts n'existera pas encore : la connexion peut réussir alors que la requête suivante échoue.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

26. Première requête SELECT

```

Using connection As New SqlConnection(connectionString)
    connection.Open()

    Dim sql As String = "SELECT Id, Name, Phone, Email FROM Contacts"
    Using command As New SqlCommand(sql, connection)
        Using reader As SqlDataReader = command.ExecuteReader()
            While reader.Read()
                Console.WriteLine(reader.GetString(1))
            End While
        End Using
    End Using
End Using

```

[QUESTION] Pourquoi une boucle While reader.Read() ? Parce qu'une requête peut renvoyer plusieurs lignes. Read avance le lecteur vers la ligne suivante et renvoie False lorsqu'il n'y en a plus.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

27. Transformer une ligne SQL en Contact

```
Dim contact As New Contact(
    reader.GetString(1),
    reader.GetString(2),
    reader.GetString(3)
)
```

```
Console.WriteLine(contact.DisplayLabel())
```

C'est une étape essentielle : la base renvoie des colonnes ; l'application préfère travailler avec des objets qui ont un sens métier. On ne fait pas encore de couche Repository, mais on montre déjà le passage « ligne de données → objet Contact ».

[POINT DE CONTRÔLE] Faire verbaliser le trajet complet : SQL renvoie une ligne → le reader lit les colonnes → New Contact construit l'objet → l'interface affiche cet objet.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

28. Requête paramétrée : rechercher sans concaténer

```
Dim sql As String =
    "SELECT Id, Name, Phone, Email " &
    "FROM Contacts WHERE Name = @name"

Using command As New SqliteCommand(sql, connection)
    command.Parameters.AddWithValue("@name", search)
    ' ... ExecuteReader
End Using
```

Le texte SQL et la valeur utilisateur sont transmis séparément. Cela évite de fabriquer une requête en collant directement le texte saisi dans la commande.

[À DIRE] Un paramètre n'est pas seulement une question de confort. C'est la manière normale de transmettre une valeur à une requête et une protection majeure contre l'injection SQL.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

29. Mauvaise version : concaténer une saisie dans SQL

```
Dim sql As String =
  "SELECT Id, Name, Phone, Email FROM Contacts " &
  "WHERE Name = '" & search & "'"
```

[PROBLÈME] Cette version mélange code SQL et donnée utilisateur. Elle devient fragile avec les apostrophes et ouvre la porte à l'injection SQL. Ne pas utiliser cette technique même pour une démo rapide.

[CORRECTION] Utiliser @name puis AddWithValue. Faire comparer visuellement les deux versions.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

30. Fonction FindContacts : version console complète

```
Imports Microsoft.Data.Sqlite

Public Function FindContacts(search As String) As List(Of Contact)
    Dim results As New List(Of Contact)()
    Dim cs As String = "Data Source=contacts.db"

    Using connection As New SqliteConnection(cs)
        connection.Open()

        Dim sql As String =
            "SELECT Name, Phone, Email FROM Contacts " &
            "WHERE Name LIKE @search ORDER BY Name"

        Using command As New SqliteCommand(sql, connection)
            command.Parameters.AddWithValue("@search", "%" & search & "%")

            Using reader As SqlDataReader = command.ExecuteReader()
                While reader.Read()
                    results.Add(New Contact(
                        reader.GetString(0),
                        reader.GetString(1),
                        reader.GetString(2)
                    ))
                End While
            End Using
        End Using
    End Using

    Return results
End Function
```

[NOTE] List(Of Contact) peut être présenté ici comme une collection redimensionnable pratique. Si le groupe est fragile, garder le focus sur « une liste de Contact » sans approfondir les génériques.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

31. Lire la fonction FindContacts par blocs

1. Créer une liste vide pour accumuler les résultats.
2. Ouvrir une connexion.
3. Préparer une requête avec un paramètre.
4. Exécuter et obtenir un lecteur.
5. Pour chaque ligne, construire un Contact.
6. Ajouter l'objet à la liste.
7. Fermer automatiquement les ressources avec Using.
8. Retourner la liste.

[QUESTION] Quelle partie dépend de la console ? Aucune. C'est volontaire : la même fonction pourra être appelée depuis WinForms.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

32. Transition console → WinForms : même logique, autre interaction

WinForms ne remplace pas la logique. Il remplace principalement la manière dont l'utilisateur fournit une saisie et déclenche une action. Le TextBox joue le rôle de Console.ReadLine ; le Button.Click joue le rôle de la commande utilisateur ; la ListBox ou le DataGridView joue le rôle de Console.WriteLine pour plusieurs résultats.

Console	WinForms
Console.ReadLine()	TextBox.Text
Menu + saisie	Bouton / contrôle
Appel direct après lecture	Événement Click
Console.WriteLine	ListBox.Items / DataGridView
Boucle d'interaction manuelle	Boucle d'événements gérée par Windows Forms

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

33. Créer un projet WinForms minimal

Dans Visual Studio : créer « Windows Forms App » en Visual Basic. Ouvrir Form1 dans le designer. Avant d'ajouter plusieurs contrôles, exécuter le formulaire vide. Les étudiants doivent voir le cycle le plus simple : projet → compilation → fenêtre.

[LIVE CODING] Ajouter un seul contrôle puis exécuter. Éviter de construire tout le formulaire dans le designer avant la première exécution.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

34. Ajouter TextBox et Button progressivement

Ajouter un TextBox nommé txtSearch et un Button nommé btnSearch. Modifier le texte du bouton en « Rechercher ». Faire exécuter avant d'écrire la logique : l'interface existe mais le clic ne produit encore aucune action.

[QUESTION] Que manque-t-il ? Une réaction au clic. Le bouton existe, mais aucun traitement n'est encore associé à l'événement.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

35. Événement Click : relier une action utilisateur au code

```
Private Sub btnSearch_Click(sender As Object, e As EventArgs) _  
    Handles btnSearch.Click  
  
    MessageBox.Show("Recherche demandée")  
End Sub
```

Faire cliquer plusieurs fois et expliquer que chaque clic déclenche l'exécution de cette procédure événementielle. Le programme WinForms reste en attente d'événements au lieu d'enchaîner les instructions puis de se terminer comme l'application console.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

36. Lire la saisie du TextBox

```
Private Sub btnSearch_Click(sender As Object, e As EventArgs) _
    Handles btnSearch.Click

    Dim search As String = txtSearch.Text.Trim()
    MessageBox.Show("Vous cherchez : " & search)
End Sub
```

[VARIATION] Tester une chaîne vide. Demander au groupe quelle validation minimale ajouter avant d'appeler la base.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

37. Validation minimale dans le formulaire

```
If String.IsNullOrEmpty(search) Then
    MessageBox.Show("Saisissez un nom à rechercher.")
    txtSearch.Focus()
    Return
End If
```

Return quitte immédiatement la procédure Click. Cette forme évite d’imbriquer toute la recherche dans un Else et garde le chemin principal plus lisible.

[POINT DE CONTRÔLE] Demander : si search est vide, la fonction FindContacts sera-t-elle appelée ? Non, car Return interrompt la procédure.

Conduite live coding de cette fiche

Construire l’interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l’écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d’ajouter la logique.
- Tester l’événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

38. Ajouter une ListBox pour les résultats

Ajouter `lstResults`. Avant d'appeler la base, injecter deux valeurs en dur afin de vérifier uniquement l'affichage. Cette technique isole les problèmes : si l'interface n'affiche pas correctement une liste connue, inutile de suspecter la base de données.

```
lstResults.Items.Clear()
lstResults.Items.Add("Alice - 0600000001")
lstResults.Items.Add("Bob - 0600000002")
```

[RÈGLE] Tester une partie simple avant de brancher une autre dépendance. Le diagnostic devient plus facile quand on sait déjà ce qui fonctionne.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du `TextBox`, `String` locale, liste de `Contact`, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

39. Brancher FindContacts au formulaire

```
Dim contacts As List(Of Contact) = FindContacts(search)
```

```
lstResults.Items.Clear()
```

```
For Each contact As Contact In contacts
```

```
    lstResults.Items.Add(contact.DisplayLabel())
```

```
Next
```

[PRÉDICTION] Si la recherche retourne 0 contact, que voit-on ? Une liste vide. Est-ce suffisant pour l'utilisateur ? Probablement non : ajoutons un message explicite.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

40. Gérer « aucun résultat »

```

If contacts.Count = 0 Then
    lstResults.Items.Add("Aucun résultat.")
    Return
End If

For Each contact As Contact In contacts
    lstResults.Items.Add(contact.DisplayLabel())
Next

```

Cette petite amélioration illustre une idée générale : une application ne doit pas seulement fonctionner techniquement ; elle doit aussi rendre son état compréhensible à l'utilisateur.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

41. Variante autonome : recherche partielle

La requête utilise LIKE avec des pourcentages. Proposer aux étudiants de prédire les résultats pour « Al », « o », puis une chaîne qui ne correspond à personne. Ensuite seulement, exécuter les trois cas.

```
command.Parameters.AddWithValue("@search", "%" & search & "%")
```

[MINI-DÉFI] Ajouter une étiquette lblCount qui affiche « 3 résultat(s) ». Ne pas donner immédiatement la solution : le nombre est disponible dans contacts.Count.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

42. Variante DataGridView : afficher plusieurs colonnes

Si le rythme du groupe le permet, remplacer ou compléter la ListBox par un DataGridView. L'objectif n'est pas d'apprendre tous les détails du binding, mais de voir qu'un même résultat objet peut être présenté autrement.

[OPTION GROUPE RAPIDE] Créer trois colonnes Name, Phone, Email et ajouter une ligne par contact. Ne pas sacrifier le temps de débogage et de tests pour cette variante.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

43. Point de synthèse : trajet complet d'une recherche

9. L'utilisateur saisit du texte dans txtSearch.
10. Le clic déclenche btnSearch_Click.
11. Le texte est validé.
12. FindContacts reçoit le critère.
13. ADO.NET ouvre la connexion.
14. Une requête paramétrée est exécutée.
15. Le reader lit les lignes.
16. Chaque ligne devient un Contact.
17. La liste est retournée au formulaire.
18. Le formulaire affiche les résultats.

[QUESTION FINALE DE LA SÉQUENCE] À quel endroit faut-il regarder si le clic ne déclenche rien ? Et si la requête renvoie 0 ligne ? Et si les lignes sont bonnes mais mal affichées ? Faire associer symptôme et zone de responsabilité.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

44. Erreurs : commencer par classer le symptôme

Type	Quand ?	Exemple
Compilation	Avant l'exécution	Nom de variable inexistant, parenthèse manquante
Exécution	Pendant le programme	Nothing utilisé comme objet, fichier/base absente
Logique	Le programme s'exécute mais le résultat est faux	Mauvais opérateur, mauvaise condition, mauvais paramètre

Le diagnostic devient plus méthodique lorsque l'on commence par demander « à quelle étape le problème apparaît ? ». Cette question réduit déjà fortement le champ de recherche.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

45. Bug 1 - erreur de compilation

Dim contacts As List(Of Contact) = FindContact(search)

[ERREUR VOLONTAIRE] Supposer que la vraie fonction s'appelle FindContacts. Compiler. Ne pas corriger immédiatement : faire lire le message, puis demander quel symbole Visual Studio ne trouve pas.

[MÉTHODE] Lire : fichier / ligne / message / nom concerné. Puis seulement corriger.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

46. Bug 2 - erreur d'exécution avec Nothing

```
Dim result As Contact = Nothing  
Console.WriteLine(result.Name)
```

[PRÉDICTION] Compilation ou exécution ? Le code compile. L'erreur n'apparaît qu'au moment où on tente d'accéder à Name.

Faire lire `NullReferenceException` et expliquer la formulation : la variable ne référence aucun objet. Éviter de mémoriser le message mot à mot ; retenir la question de diagnostic : « quelle référence vaut Nothing ? ».

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

47. Bug 3 - erreur de logique

Dim sql As String =

"SELECT Name, Phone, Email FROM Contacts WHERE Name LIKE @search"

command.Parameters.AddWithValue("@search", search)

La recherche partielle ne fonctionne plus si l'on oublie les caractères %. Le programme ne plante pas ; il renvoie seulement moins de résultats que prévu. C'est typiquement une erreur de logique.

[DIAGNOSTIC] Comparer donnée saisie, valeur réelle du paramètre et intention. Le débogueur va précisément nous permettre d'inspecter ces valeurs.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

48. Méthode générale de diagnostic en 6 questions

19. Quel est le symptôme observable ?
20. À quel moment apparaît-il : compilation, démarrage, clic, requête, affichage ?
21. Quel était le résultat attendu ?
22. Quelle est la dernière étape dont on sait qu'elle fonctionne ?
23. Quelle valeur ou condition peut expliquer l'écart ?
24. Comment vérifier cette hypothèse par une observation plutôt que par une supposition ?

[RÈGLE] Le débogage n'est pas « modifier jusqu'à ce que ça marche ». C'est formuler une hypothèse puis la vérifier.

Conduite live coding de cette fiche

Commencer par faire reformuler le besoin métier avant de prononcer le vocabulaire objet. Demander ensuite au groupe de prédire ce que le code autorise déjà et ce qu'il n'autorise pas encore. Taper une seule nouveauté à la fois, compiler, puis demander aux étudiants de décrire ce qui vient de changer dans le modèle. L'objectif est de relier en permanence la syntaxe Visual Basic à une représentation mentale : un type défini par le développeur, des instances distinctes, des données regroupées et des comportements placés au bon endroit.

- Faire nommer explicitement la classe et l'instance utilisées dans l'exemple.
- Demander quelle responsabilité appartient réellement à Contact et laquelle appartient à l'interface.
- Terminer par une petite variation : nouvelle propriété, nouvelle instance ou appel de méthode, puis exécuter.

49. Préparer une recherche volontairement incorrecte

Pour la séquence débogueur, utiliser un bug reproductible : la recherche de « Bob » renvoie 0 résultat alors qu'il existe. Exemple : le code ajoute accidentellement un espace à la fin du critère.

```
Dim search As String = txtSearch.Text & " "  
Dim contacts As List(Of Contact) = FindContacts(search)
```

[QUESTION] Comment prouver que search contient réellement cet espace ? Ne pas répondre « en lisant le code » : nous allons l'observer à l'exécution.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

50. Breakpoint : arrêter le programme au bon endroit

Placer un point d'arrêt sur la ligne qui appelle FindContacts. Lancer le formulaire, saisir Bob, cliquer. Le programme s'arrête avant l'exécution de la ligne. Expliquer que le breakpoint suspend le programme sans le terminer et permet d'inspecter son état.

[POINT DE CONTRÔLE] Les étudiants doivent distinguer : arrêt du programme, pause au breakpoint, et erreur. Un breakpoint n'est pas un problème : c'est un outil volontaire.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

51. Locals : observer les variables déjà disponibles

Ouvrir la fenêtre Locals. Repérer search. Faire lire sa valeur avec les guillemets si l'interface les montre. Demander si la valeur correspond exactement à la prédiction « Bob ».

[PRÉDICTION] Avant d'inspecter : Bob ou « Bob » avec un espace ? Puis vérifier. Cette micro-prédiction transforme l'outil de débogage en expérience.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

52. Watch : suivre précisément une expression

Ajouter search à Watch. Ajouter aussi search.Length. On obtient une information plus objective : si Bob devrait avoir une longueur de 3 et que la valeur observée est 4, quelque chose a été ajouté.

[VARIATION] Ajouter txtSearch.Text.Length puis comparer avec search.Length. Cette comparaison localise la transformation fautive entre la saisie et la variable.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

53. Step Over : exécuter une ligne sans entrer dans la fonction

Step Over exécute la ligne courante puis s'arrête à la suivante dans le même niveau. Utiliser cet outil quand on considère la fonction appelée comme une boîte noire momentanément et que l'on veut observer le résultat global.

[QUESTION] Si nous sommes sur FindContacts(search) et faisons Step Over, où allons-nous ? À la ligne suivante du Click, une fois la fonction terminée.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

54. Step Into : entrer dans FindContacts

Recommencer et choisir Step Into. Visual Studio entre dans FindContacts. Faire suivre le paramètre search. L'objectif est de montrer que l'on peut descendre dans l'appel lorsque l'hypothèse concerne le fonctionnement interne de la fonction.

[RÈGLE DE CHOIX] Step Over si l'appel n'est probablement pas en cause ; Step Into si l'on veut comprendre ou diagnostiquer ce qui se passe à l'intérieur.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

55. Step Out : ressortir d'une fonction

Lorsque l'on a suffisamment inspecté FindContacts, Step Out termine la fonction courante et revient à son appelant. C'est utile pour éviter de pas-à-pas chaque ligne lorsque l'hypothèse vient d'être vérifiée ou réfutée.

[MINI-DÉFI] Faire entrer dans FindContacts, inspecter le paramètre, puis ressortir sans parcourir toute la lecture SQL.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

56. Observer le paramètre SQL réel

Placer un breakpoint juste avant `ExecuteReader`. Inspecter `command.Parameters("@search").Value`. Le groupe peut maintenant comparer la valeur envoyée à la base avec ce qui était attendu.

[DIAGNOSTIC] Si la valeur contient « %Bob % » au lieu de « %Bob% », la base ne fait que répondre à la requête reçue. Le bug est en amont, dans la construction de `search`.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

57. Corriger puis rejouer exactement le même scénario

Supprimer l'espace ajouté, relancer et rechercher Bob avec les mêmes étapes. Un bon diagnostic se termine par une vérification reproductible du scénario qui échouait. Ne pas se contenter de « ça a l'air bon ».

[POINT DE CONTRÔLE] Symptôme initial : 0 résultat. Cause : critère modifié. Correction : Trim ou suppression de l'espace. Vérification : même recherche, résultat Bob affiché.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

58. Pile d'appels : comprendre « comment suis-je arrivé ici ? »

La Call Stack montre la chaîne d'appels active. Dans notre scénario : btnSearch_Click → FindContacts → ExecuteReader / lecture. Pour des débutants, l'objectif est simplement de comprendre que plusieurs fonctions sont imbriquées et que l'erreur peut être observée loin de l'endroit où la mauvaise valeur a été créée.

[TABLEAU] Dessiner une pile simple : Click en bas, FindContacts au-dessus. Montrer que revenir dans la pile aide à chercher l'origine d'une valeur.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

59. Exceptions : lire les informations utiles

Lorsqu'une exception apparaît, apprendre à chercher quatre éléments : type de l'exception, message, ligne mise en évidence, pile d'appels. Il n'est pas nécessaire de comprendre toutes les lignes internes de .NET.

[RÈGLE] Commencer par la première ligne de votre propre code indiquée dans la pile, pas par les détails internes du framework.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

60. Bug préparé : base absente ou mauvais chemin

Modifier temporairement la chaîne de connexion vers un chemin incorrect ou déplacer contacts.db. L'objectif est de montrer qu'un bug peut être extérieur à l'algorithme : fichier introuvable, table absente, configuration incorrecte.

[DIAGNOSTIC] Questionner : la classe Contact est-elle responsable ? Non. Le problème apparaît au moment de l'accès aux données. On resserre donc la recherche sur connexion / chemin / schéma.

Conduite live coding de cette fiche

Ne pas laisser la syntaxe ADO.NET masquer le trajet de la donnée. Avant chaque nouvelle ligne, faire pointer sur le schéma « application → connexion → commande → base → reader → Contact ». Faire prédire la valeur du paramètre et le nombre de lignes attendu avant l'exécution. Après l'exécution, revenir à ce qui a réellement été observé. Si le résultat est vide, résister à la tentation de modifier immédiatement le SQL : inspecter d'abord le critère, le chemin de base et le paramètre transmis.

- Vérifier la base avec des données connues avant de coder la recherche.
- Faire comparer concaténation SQL et paramètre sans dramatiser : montrer le problème concret.
- Finir par raconter oralement le trajet d'une ligne SQL jusqu'à un objet Contact.

61. Test unitaire : pourquoi ajouter encore un outil ?

Le débogueur aide à comprendre une exécution précise. Un test unitaire automatise une vérification que l'on veut pouvoir rejouer souvent. Il est particulièrement utile pour les fonctions pures ou quasi pures : mêmes entrées, résultat prévisible, pas de dépendance à l'interface ou à la base.

[TRANSITION] Choisir une petite fonction déjà produite en J2 ou J3, par exemple `IsValidEmail` ou `NormalizeName`. Ne pas commencer par tester WinForms ou SQLite.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

62. Fonction cible de test : IsEmailValid

```
Public Function IsEmailValid(email As String) As Boolean
    Return Not String.IsNullOrEmpty(email) AndAlso
        email.Contains("@")
End Function
```

Cette validation est volontairement simplifiée. Le but n'est pas de couvrir toute la norme des adresses email, mais d'avoir une règle déterministe et facile à tester.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

63. Arrange - Act - Assert

Phase	Question	Dans notre exemple
Arrange	Quelles données prépare-t-on ?	email = "alice@example.com"
Act	Quelle action teste-t-on ?	IsValidEmail(email)
Assert	Quel résultat attend-on ?	True

AAA structure le test comme une petite expérience : préparer, agir, vérifier.

[QUESTION] Où met-on Console.WriteLine dans AAA ? Nulle part : le test vérifie directement une valeur attendue.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

64. Premier test MSTest

```
Imports Microsoft.VisualStudio.TestTools.UnitTesting
```

```
<TestClass>
```

```
Public Class ContactTests
```

```
<TestMethod>
```

```
Public Sub IsEmailValid_WithAt_ReturnsTrue()
```

```
    ' Arrange
```

```
    Dim email As String = "alice@example.com"
```

```
    ' Act
```

```
    Dim result As Boolean = IsEmailValid(email)
```

```
    ' Assert
```

```
    Assert.IsTrue(result)
```

```
End Sub
```

```
End Class
```

[EXÉCUTION] Lancer le test depuis Test Explorer et faire identifier l'état vert. Vert signifie : pour ce scénario précis, le résultat observé correspond à l'attendu.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

65. Faire passer le test au rouge volontairement

Modifier temporairement la fonction pour qu'elle renvoie False dans tous les cas. Relancer exactement le même test.

```
Public Function IsValid(email As String) As Boolean
    Return False
End Function
```

[PRÉDICTION] Le test compile-t-il ? Oui. Passe-t-il ? Non. Le rouge n'est pas une erreur du framework : c'est une différence entre résultat attendu et résultat obtenu.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

66. Lire un échec de test

Faire lire le nom du test, le message d'assertion, la valeur attendue et la valeur obtenue. Relier cette lecture à la méthode de diagnostic utilisée plus tôt : symptôme, attendu, obtenu, hypothèse, correction.

[RÈGLE] Un test utile doit échouer pour une bonne raison lorsqu'on casse le comportement qu'il protège.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

67. Corriger le code et revenir au vert

```
Public Function IsValid(email As String) As Boolean
    Return Not String.IsNullOrEmpty(email) AndAlso
        email.Contains("@")
End Function
```

[EXÉCUTION] Relancer le test sans le modifier. Le retour au vert montre que le comportement attendu est restauré.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

68. Cas limite : chaîne vide

```
<TestMethod>
Public Sub IsEmailValid_Empty_ReturnsFalse()
    ' Arrange
    Dim email As String = ""

    ' Act
    Dim result As Boolean = IsEmailValid(email)

    ' Assert
    Assert.IsFalse(result)
End Sub
```

Expliquer la différence entre cas nominal et cas limite. Un seul test vert ne prouve pas que la fonction est correcte pour toutes les entrées.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

69. Test et régression

Une régression est un comportement qui fonctionnait puis cesse de fonctionner après une modification. Garder un test dans le projet permet de rejouer automatiquement le même scénario après chaque changement important.

[ANALOGIE] Le test est une alarme réutilisable : il ne garantit pas que toute la maison est sûre, mais il surveille explicitement un comportement.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

70. Ce qu'il ne faut pas tester aujourd'hui

Pour maintenir le niveau débutant et le temps disponible, ne pas ouvrir le chantier des tests d'interface, mocks, dépendances, intégration de base de données ou architecture de test avancée. Le message pédagogique est simple : une petite fonction peut être vérifiée automatiquement.

[LIMITE DU COURS] MSTest est présenté comme une première porte d'entrée, pas comme un cours complet de stratégie de tests.

Conduite live coding de cette fiche

Présenter le test comme une expérience reproductible. Les étudiants doivent savoir quelle entrée est préparée, quelle action est appelée et quel résultat est attendu avant de cliquer sur Run. Faire volontairement casser la fonction pour démontrer qu'un bon test devient rouge lorsque le comportement protégé est rompu. Le retour au vert n'a de sens qu'après lecture du message d'échec et correction du code de production.

- Nommer explicitement Arrange, Act et Assert dans le premier test.
- Faire prédire vert ou rouge avant chaque exécution.
- Relier le test à un comportement métier précis plutôt qu'à une ligne de code.

71. TP 3 - objectif global

Le TP consiste à obtenir un formulaire de recherche de contacts fonctionnel : l'utilisateur saisit un texte, clique sur Rechercher, l'application interroge une base locale avec une requête paramétrée, transforme les lignes en objets Contact et affiche les résultats. Une variante ajoute un message « aucun résultat ».

- Créer ou ouvrir le projet WinForms fourni.
- Vérifier que la base contacts.db est disponible.
- Créer/reliure Contact.vb.
- Implémenter FindContacts.
- Créer txtSearch, btnSearch et lstResults.
- Coder le Click.
- Tester au moins trois recherches.
- Diagnostiquer un bug injecté.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

72. TP 3 - niveaux d'aide progressifs

Niveau	Aide
Indice 1	Le clic doit d'abord lire txtSearch.Text.Trim().
Indice 2	La fonction de recherche retourne une List(Of Contact).
Indice 3	La requête utilise WHERE Name LIKE @search.
Indice 4	Le paramètre reçoit "%" & search & "%".
Indice 5	Pour chaque Contact retourné, ajouter DisplayLabel() dans la ListBox.

Ne donner l'indice suivant que si l'étudiant a réellement tenté quelque chose. L'aide progressive préserve l'activité de raisonnement.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

73. TP 3 - correction étape 1 : formulaire et validation

```
Private Sub btnSearch_Click(sender As Object, e As EventArgs) _
    Handles btnSearch.Click

    Dim search As String = txtSearch.Text.Trim()

    If String.IsNullOrEmpty(search) Then
        MessageBox.Show("Saisissez un nom.")
        txtSearch.Focus()
        Return
    End If
End Sub
```

[QUESTION] Pourquoi commencer par cette partie avant la base ? Parce qu'elle peut être testée immédiatement et isole la responsabilité d'interaction.

Conduite live coding de cette fiche

Construire l'interface comme le code : incrémentalement. Ajouter un contrôle, exécuter, observer, puis seulement ajouter le suivant. Faire distinguer ce qui est visible à l'écran de ce qui constitue la logique. Le clic doit être présenté comme un événement qui déclenche une procédure, non comme une magie du designer. À chaque étape, demander où se trouve la donnée et sous quelle forme : texte du TextBox, String locale, liste de Contact, éléments affichés.

- Exécuter le formulaire vide avant d'ajouter la logique.
- Tester l'événement avec un message simple avant de brancher la base.
- Faire varier une saisie vide, un résultat unique et aucun résultat.

74. TP 3 - correction étape 2 : appel de recherche

```
Dim contacts As List(Of Contact) = FindContacts(search)
```

```
lstResults.Items.Clear()
```

```
If contacts.Count = 0 Then
```

```
    lstResults.Items.Add("Aucun résultat.")
```

```
    Return
```

```
End If
```

Exécuter maintenant avec une version temporaire de FindContacts qui retourne une liste connue si nécessaire. Le but est de tester progressivement l'interface avant de dépendre entièrement de SQLite.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

75. TP 3 - correction étape 3 : affichage

```
For Each contact As Contact In contacts
    lstResults.Items.Add(contact.DisplayLabel())
Next
```

[VARIATION] Afficher uniquement Name et Email. Puis revenir à DisplayLabel(). Faire constater qu'une méthode de Contact permet de centraliser le format.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

76. TP 3 - correction complète : Contact.vb

```
Public Class Contact
    Public Property Name As String
    Public Property Phone As String
    Public Property Email As String

    Public Sub New(name As String, phone As String, email As String)
        Me.Name = name.Trim()
        Me.Phone = phone.Trim()
        Me.Email = email.Trim()
    End Sub

    Public Function DisplayLabel() As String
        Return Name & " - " & Phone & " - " & Email
    End Function
End Class
```

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

77. TP 3 - correction complète : ContactData.vb

```
Imports Microsoft.Data.Sqlite

Public Module ContactData
    Private Const ConnectionString As String = "Data Source=contacts.db"

    Public Function FindContacts(search As String) As List(Of Contact)
        Dim results As New List(Of Contact)()

        Using connection As New SqliteConnection(ConnectionString)
            connection.Open()

            Dim sql As String =
                "SELECT Name, Phone, Email FROM Contacts " &
                "WHERE Name LIKE @search ORDER BY Name"

            Using command As New SqliteCommand(sql, connection)
                command.Parameters.AddWithValue("@search", "%" & search & "%")

                Using reader As SqliteDataReader = command.ExecuteReader()
                    While reader.Read()
                        results.Add(New Contact(
                            reader.GetString(0),
                            reader.GetString(1),
                            reader.GetString(2)
                        ))
                    End While
                End Using
            End Using
        End Using

        Return results
    End Function
End Module
```

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

78. TP 3 - correction complète : Form1.vb

```
Public Class Form1
    Private Sub btnSearch_Click(sender As Object, e As EventArgs) _
        Handles btnSearch.Click

        Dim search As String = txtSearch.Text.Trim()

        If String.IsNullOrEmpty(search) Then
            MessageBox.Show("Saisissez un nom à rechercher.")
            txtSearch.Focus()
            Return
        End If

        Dim contacts As List(Of Contact) = FindContacts(search)
        lstResults.Items.Clear()

        If contacts.Count = 0 Then
            lstResults.Items.Add("Aucun résultat.")
            Return
        End If

        For Each contact As Contact In contacts
            lstResults.Items.Add(contact.DisplayLabel())
        Next
    End Sub
End Class
```

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

79. Arborescence finale conseillée

```
ContactsApp/  
├─ Contact.vb  
├─ ContactData.vb  
├─ Form1.vb  
├─ Form1.Designer.vb  
├─ contacts.db  
└─ ContactsApp.vbproj
```

Cette organisation reste volontairement simple. Contact représente le métier, ContactData contient l'accès SQLite, Form1 gère l'interaction. Elle suffit pour rendre visibles les responsabilités sans introduire une architecture plus avancée.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

80. Checklist de diagnostic pour le formateur et les étudiants

- Le projet compile-t-il ? Si non : lire l'erreur de compilation.
- Le clic est-il déclenché ? Mettre un breakpoint dans btnSearch_Click.
- search contient-il la valeur attendue ? Locals / Watch.
- FindContacts est-elle appelée ? Step Into.
- La chaîne de connexion pointe-t-elle vers le bon fichier ?
- Le paramètre SQL contient-il les % attendus ?
- reader.Read() renvoie-t-il au moins une ligne ?
- Les objets Contact sont-ils créés avec les bonnes colonnes ?
- La liste retournée contient-elle des éléments ?
- La ListBox est-elle vidée puis remplie ?

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmar, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

81. Questions fréquentes des étudiants - réponses courtes

Question	Réponse à donner
Pourquoi une classe si un tableau fonctionne ?	Parce qu'une classe représente explicitement une chose du domaine et regroupe ses données.
Pourquoi New ?	Pour créer une instance et exécuter son constructeur.
Pourquoi Using ?	Pour libérer automatiquement la ressource à la fin du bloc.
Pourquoi @search ?	Pour transmettre une valeur séparément du SQL.
Pourquoi un breakpoint ?	Pour suspendre l'exécution et observer l'état réel.
Pourquoi tester une Function ?	Pour vérifier automatiquement un comportement sur des entrées connues.
Pourquoi ContactData séparé ?	Pour éviter que le formulaire contienne toute la logique d'accès aux données.

Conduite live coding de cette fiche

Pour cette étape, conserver le rythme court du cours : poser une question, obtenir une prédiction, réaliser une action visible dans Visual Studio, exécuter immédiatement et demander ce que l'observation confirme ou contredit. La notion doit être reliée au fil rouge Contacts afin que les étudiants comprennent pourquoi elle apparaît maintenant et non comme une syntaxe isolée.

- Une seule idée nouvelle par modification.
- Une exécution ou observation après chaque modification significative.
- Une mini-variation confiée au groupe avant de passer à la suite.

82. Erreurs fréquentes à provoquer ou surveiller

Erreur	Symptôme	Piste pédagogique
Oubli de New	Variable Nothing ou constructeur non appelé	Revenir à classe vs instance.
Nom de propriété incorrect	Compilation	Lire le membre introuvable.
Résultat Nothing utilisé	NullReferenceException	Tester Is Nothing.
Mauvais index reader	Donnée incohérente / exception	Associer ordre des colonnes et GetString.
SQL concaténé	Fragilité / sécurité	Revenir aux paramètres.
Mauvais chemin .db	Erreur accès / table absente	Inspecter chaîne de connexion.
Event handler non relié	Clic sans effet	Vérifier Handles btnSearch.Click.
Liste non vidée	Résultats qui s'accumulent	Clear avant affichage.
Mauvais critère LIKE	0 résultat	Inspecter paramètre avec Watch.

Conduite live coding de cette fiche

Ici, la compétence travaillée n'est pas de connaître tous les boutons du débogueur : c'est d'adopter une démarche. Toujours demander un résultat attendu avant de lancer, puis faire décrire le symptôme sans interprétation. Formuler une hypothèse, choisir une observation qui peut la confirmer ou l'infirmier, puis utiliser le débogueur. Le formateur doit volontairement ralentir la correction : un bug résolu trop vite par celui qui connaît la réponse n'enseigne presque rien.

- Faire verbaliser l'hypothèse avant tout Step Into ou Watch.
- Comparer la valeur prédite à la valeur réellement observée.
- Après correction, rejouer exactement le scénario qui échouait.

83. Synthèse conceptuelle du J3

Besoin	Notion introduite
Regrouper les données d'un contact	Classe / objet / propriétés
Initialiser correctement un contact	Constructeur Sub New
Donner un comportement local	Méthode
Conserver les contacts	Base SQLite
Communiquer avec la base	ADO.NET
Rechercher sans concaténer	Requête paramétrée
Déclencher par un clic	WinForms / événement
Comprendre un défaut	Debugger / breakpoints / Watch
Sécuriser une règle	Test unitaire MSTest

[QUESTION] Faire reformuler chaque ligne par un étudiant différent. On cherche le lien besoin → outil, pas la récitation d'une syntaxe.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

84. Évaluation finale - mini-cas intégrateur

Donner un projet dont la recherche de contacts présente deux défauts : un bug de logique dans le paramètre LIKE et une fonction IsEmailValid volontairement cassée. Demander aux étudiants de : expliquer la classe Contact, exécuter la recherche, localiser le bug avec un breakpoint, corriger, puis relancer un test MSTest fourni.

Critère	Preuve observable
Objet	Explique classe, instance, propriété, constructeur
Données	Décrit connexion → commande → reader → Contact
Interface	Explique le rôle du Click et de txtSearch
Diagnostic	Formule une hypothèse puis utilise le debugger
Test	Explique pourquoi le test est rouge puis ce qui le rend vert

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

85. Correction commentée du mini-cas intégrateur

25. Vérifier la saisie dans txtSearch.
26. Placer un breakpoint avant FindContacts.
27. Observer search dans Watch.
28. Entrer dans FindContacts.
29. Contrôler la valeur de @search.
30. Corriger la construction du paramètre LIKE.
31. Rejouer le même scénario et vérifier les résultats.
32. Ouvrir Test Explorer.
33. Lancer IsEmailValid_WithAt_ReturnsTrue.
34. Lire l'échec, corriger la fonction, relancer.

[RÈGLE] La correction doit reproduire la démarche, pas seulement montrer la bonne ligne finale.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

86. Adaptation - groupe moins expérimenté

- Rester plus longtemps sur classe / objet / propriétés avant le constructeur.
- Donner Contact.vb précréé et faire compléter seulement les propriétés.
- Fournir contacts.db prêt avec les données.
- Limiter la requête à SELECT exact avant LIKE.
- Utiliser ListBox plutôt que DataGridView.
- Réduire le nombre de bugs injectés à deux.
- Faire un seul test unitaire et commenter clairement AAA.

[PRIORITÉ] Si le temps manque, préserver classe Contact, requête paramétrée, formulaire minimal et débogueur. La variante DataGridView et les approfondissements peuvent disparaître.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

87. Adaptation - groupe plus expérimenté

- Remplacer le tableau par List(Of Contact) plus tôt.
- Ajouter Try/Catch autour de l'accès aux données et discuter ce qu'il faut réellement gérer.
- Introduire une classe ContactRepository ou ContactDataService uniquement comme prolongement.
- Ajouter recherche par email ou téléphone.
- Ajouter un test sur NormalizeName et un test de cas limite.
- Faire refactoriser le Click pour réduire sa taille.
- Comparer deux manières d'afficher les résultats.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

88. Glossaire J3

Terme	Définition simple
Classe	Type créé par le développeur pour représenter une chose et ses comportements.
Objet / instance	Valeur concrète créée à partir d'une classe.
Propriété	Donnée accessible sur un objet.
Méthode	Traitement appartenant à un type ou un objet.
Constructeur	Procédure exécutée lors de New.
Encapsulation	Idée de regrouper et protéger l'état et les règles d'un objet.
ADO.NET	Ensemble d'API .NET pour accéder à des sources de données.
Connexion	Objet représentant le lien avec la base.
Commande	Objet représentant une requête à exécuter.
Reader	Objet permettant de lire les lignes renvoyées.
Requête paramétrée	Requête dont les valeurs sont fournies séparément du SQL.
Événement	Fait auquel l'interface peut réagir, par exemple un clic.
Breakpoint	Point où le débogueur suspend volontairement l'exécution.
Watch	Fenêtre permettant de suivre une expression.
Test unitaire	Vérification automatisée d'un petit comportement isolé.
Régression	Comportement qui fonctionnait puis casse après une modification.

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

89. Quick reference - syntaxe essentielle

```
Public Class Contact
    Public Property Name As String

    Public Sub New(name As String)
        Me.Name = name
    End Sub

    Public Function DisplayLabel() As String
        Return Name
    End Function
End Class

Using connection As New SqlConnection(connectionString)
    connection.Open()
End Using

Private Sub btnSearch_Click(sender As Object, e As EventArgs) _
    Handles btnSearch.Click
End Sub

<TestClass>
Public Class ContactTests
    <TestMethod>
    Public Sub Example()
        Assert.IsTrue(True)
    End Sub
End Class
```

Conduite live coding de cette fiche

Utiliser cette fiche comme un point de consolidation plutôt que comme un exposé. Faire travailler les étudiants d'abord, puis demander plusieurs propositions avant d'afficher la correction. Lors du débrief, privilégier les règles de décision : « comment ai-je choisi ? », « comment ai-je vérifié ? », « quel symptôme m'a orienté ? ». La correction doit être rejouée dans Visual Studio lorsque le code est central, afin de conserver l'esprit live coding jusqu'à la fin.

- Demander une reformulation sans lire le support.
- Faire justifier au moins un choix de structure ou de diagnostic.
- Conserver une trace exécutable propre après la correction.

90. Checklist de fin de journée

- Je sais expliquer la différence entre classe et objet.
- Je sais créer Contact.vb avec propriétés et constructeur.
- Je sais créer plusieurs instances avec New.
- Je sais expliquer pourquoi une méthode peut retourner une valeur plutôt qu'afficher directement.
- Je sais décrire connexion → commande → reader → Contact.
- Je sais expliquer l'intérêt d'une requête paramétrée.
- Je sais relier un Button.Click à une recherche.
- Je sais distinguer compilation, exécution et logique.
- Je sais utiliser un breakpoint et inspecter une variable.
- Je sais expliquer Arrange-Act-Assert et interpréter un test vert/rouge.

[POINT DE CONTRÔLE J3] L'apprenant sait expliquer sa classe Contact, décrire le trajet d'une recherche jusqu'à la base, utiliser le débogueur pour localiser un défaut et expliquer ce qu'un test unitaire protège.

À la fin du J3, revenir au fil rouge complet et demander au groupe de raconter l'application sans regarder le code : de la saisie utilisateur jusqu'à la base, puis du résultat jusqu'à l'écran. Cette verbalisation finale vérifie que les notions ne sont pas restées isolées.